UNIVERSIDADE FEDERAL DE OURO PRETO
INSTITUTO DE CIÊNCIAS EXATAS E BIOLÓGICAS
DEPARTAMENTO DE COMPUTAÇÃO

MATHEUS PEIXOTO RIBEIRO VIEIRA
Orientador: Pedro Henrique Lopes Silva

DETECÇÃO DE DEEPFAKES EM VÍDEO

Ouro Preto, MG
2025

UNIVERSIDADE FEDERAL DE OURO PRETO
INSTITUTO DE CIÊNCIAS EXATAS E BIOLÓGICAS
DEPARTAMENTO DE COMPUTAÇÃO

MATHEUS PEIXOTO RIBEIRO VIEIRA

DETECÇÃO DE DEEPFAKES EM VÍDEO

Monografia apresentada ao Curso de Ciência da Computação da Universidade Federal de Ouro Preto como parte dos requisitos necessários para a obtenção do grau de Bacharel em Ciência da Computação.

Orientador: Pedro Henrique Lopes Silva

Ouro Preto, MG
2025

Dedico este trabalho aos meus pais, Cláudia e Omar, e à minha irmã, Camila.

Agradecimentos

Agradeço aos meus pais, Cláudia e Omar, por todo o apoio e incentivo que me permitiram superar os mais diversos desafios desta jornada. Vocês foram a base sólida sobre a qual todo este trabalho pôde ser construído, sempre próximos e dispostos a me apoiar nos momentos em que mais tive dúvidas e incertezas.

Agradeço à minha irmã, Camila, por me lembrar de que existe vida além das tarefas e por me mostrar a importância de criar memórias que serão guardadas e revisitadas por toda a vida.

Agradeço a meus primos Caio Henrique e Luís Felipe por me mostrarem os caminhos da computação e o passos necessários para iniciar minha trajetória. Também agradeço imensamente aos meus primos Thales e Thaciane, que me ensinaram, e muito, a como enfrentar desafios, cultivar relacionamentos e, sobretudo, buscar equilíbrio.

Agradeço aos amigos que conheci ao longo desta jornada e que desejo levar para a vida. Muito obrigado pelas incontáveis horas de conversa, risadas, aprendizados e reflexões sobre a vida e o futuro, seja em casa, na sala de aula, no CSILab, no GAID, no RU ou em qualquer outro espaço, físico ou virtual. Não consigo imaginar o quanto essa caminhada teria sido mais desafiadora sem a presença de cada um de vocês. Por isso, deixo meus mais sinceros agradecimentos a Bruno Braga, Felipe Braz, Gabriel Saldanha, Lucas Chagas, Lucas Silva, Nicolas Mendes, Pedro Henrique, Pedro Morais e Vitor Oliveira.

Agradeço ao meu orientador, Prof. Dr. Pedro Silva, por todo o suporte e ensinamento ao longo destes anos, e por me permitir ingressar em uma área de estudos pela qual sempre tive curiosidade, mas que antes parecia difícil, distante e nebulosa. Hoje, graças à sua orientação, essa área se tornou mais leve, clara e inspiradora para o futuro.

Por fim, gostaria de agradecer ao meu professor do IFMG – *campus* Ouro Branco, Carlos Eduardo Paulino, que foi fundamental no início do meu aprendizado em programação e que me incentivou a seguir na área da computação. Estendo também meus agradecimentos a todos os professores do DECOM que fizeram parte da minha trajetória, tanto àqueles com quem tive o privilégio de aprender em sala de aula, quanto àqueles que conheci e com quem conversei apenas nos corredores do ICEB e pela UFOP.

“É justamente a possibilidade de realizar um sonho que torna a vida interessante”
Coelho (2001)

Resumo

Este trabalho aborda a detecção automática de vídeos manipulados por técnicas de deepfake, um tema de crescente relevância diante do uso malicioso dessas tecnologias em contextos políticos, sociais e financeiros. Trata-se de uma pesquisa experimental que propõe e avalia um modelo de rede neural baseado na arquitetura Xception, adaptado para sequências de vídeo, com foco em alcançar desempenho competitivo aliado a menor custo computacional. A metodologia adotada inclui o uso do dataset Celeb-DF v2, aplicação de pré-processamento com extração facial via RetinaFace, e um pipeline de classificação que utiliza as camadas TimeDistributed e GlobalAveragePooling3D. Foram testadas diferentes combinações de hiperparâmetros por meio de otimização bayesiana, com o melhor modelo atingindo acurácia de 97,30% e AUC de 0,9957, utilizando apenas 24 quadros por vídeo. Os resultados demonstram que, mesmo com menos informações temporais, o modelo proposto supera abordagens mais complexas em eficiência e apresenta desempenho comparável a métodos de estado da arte. Conclui-se que é possível desenvolver soluções robustas e acessíveis para a detecção de deepfakes, contribuindo para o combate à desinformação e fraudes digitais.

Palavras-chave: *Deepfake*. Redes Neurais. *Deep learning*. Xception.

Abstract

This work addresses the automatic detection of videos manipulated by deepfake techniques, a topic of growing concern given the malicious use of such technologies in political, social, and financial contexts. It is an experimental research project that proposes and evaluates a neural network model based on the Xception architecture, adapted for video sequences, focusing on achieving competitive performance with reduced computational cost. The methodology includes the use of the Celeb-DF v2 dataset, facial region extraction using RetinaFace, and a classification pipeline that integrates TimeDistributed and GlobalAveragePooling3D layers. Various hyperparameter combinations were explored through Bayesian optimization, with the best-performing model achieving an accuracy of 97.30% and an AUC of 0.9957 using only 24 frames per video. The results demonstrate that, even with fewer temporal features, the proposed model outperforms more complex approaches in computational efficiency while maintaining performance comparable to state-of-the-art methods. It is concluded that robust and accessible solutions for deepfake detection are feasible, contributing to the fight against misinformation and digital fraud.

Keywords: Deepfake, Neural Networks, Deep Learning. Xception.

Lista de Ilustrações

Figura 2.1 – Adaptação do perceptron de (ROSENBLATT, 1958). 7
Figura 2.2 – MLP com uma camada de entrada, duas ocultas e uma de saída 8
Figura 2.3 – Função ReLU 10
Figura 2.4 – Representação da computação do erro de uma rede neural 14
Figura 2.5 – Representação da computação do erro de uma rede neural 15
Figura 2.6 – Operação de convolução 16
Figura 2.7 – Diferentes filtros convolucionais aprendidos e aplicados para a detecção de um gato 18
Figura 2.8 – Operação de convolução com múltiplas camadas de entrada 18
Figura 2.9 – Fluxograma de uma Rede Neural Adversarial 19
Figura 2.10–Comparação de imagem original e imagem com aumento de dados 21
Figura 2.11–Exemplo de aplicação do *dropout*. 22
Figura 2.12–*Overfitting* do modelo. 23
Figura 2.13–Matriz de confusão 25
Figura 2.14–Curva ROC 26
Figura 2.15–Imagem real à esquerda e *deepfake* à direita 27
Figura 3.1 – Imagem real do conjunto Celeb-real. 32
Figura 3.2 – Imagem de uma pessoa real do conjunto Youtube-real. 32
Figura 3.3 – Imagem de um vídeo com *deepfake* do conjunto Celeb-synthesis. 33
Figura 3.4 – Imagem do *dataset* com resolução 450×500. 36
Figura 3.5 – Imagem com o rosto extraído. 36
Figura 3.6 – Imagem com o rosto extraído e borda preta 36
Figura 3.7 – Imagem original à esquerda e rosto extraído à direita. 37
Figura 3.8 – Arquitetura do modelo Xception. 38
Figura 3.9 – Modelo Final proposto. 39
Figura 4.1 – Resultados obtidos durante 15 épocas 41
Figura 4.2 – Erros obtidos 42
Figura 4.3 – Acurácias obtidas 42
Figura 4.4 – AUC obtidos 43
Figura 4.5 – Matriz de confusão dos dados de teste 44
Figura 4.6 – Reais classificados como falsos. 44
Figura 4.7 – Falsos classificados como reais 45

Figura 4.8 – Mapas de ativação do primeiro quadro (primeira linha), do quadro intermediário (segunda linha, décimo segundo quadro) e do quadro final de quatro vídeos e suas respectivas classificações. Regiões com maior relevância estão destacadas em vermelho, enquanto áreas menos significativas são representadas em azul. 45

Lista de Tabelas

Tabela 2.1 – Resultado dos trabalhos relacionados. 31
Tabela 3.1 – Informações da quantidade de quadros dos vídeos. 33
Tabela 3.2 – Divisão dos vídeos. 35
Tabela 4.1 – Resultados obtidos durante as 15 épocas de treinamento. 41
Tabela 4.2 – Comparação do modelo proposto com trabalhos que utilizaram a Celeb-DF v2, ordenados pelo AUC 43
Tabela 5.1 – Separação de atividades para a Monografia II 47

Lista de Abreviaturas e Siglas

AUC *Area Underneath the Curve*. vii, ix, 2, 26, 28–31, 39, 41–43, 46

BCE *Binary Cross-Entropy Loss*. 12, 18

CNN *Convolutional Neural Network*. 1, 16, 20, 28, 29, 46

FN *False negative*. 25

FP *False positive*. 25, 35

GAN *Generative Adversarial Networks*. 18, 27

LSTM *Long Short-term Memory*. 28

MAE *Mean Absolute Error*. 12

MLP *Multilayer perceptron*. vii, 8

MSE *Mean Absolute Error*. 12

PCA *Principal Component Analysis*. 30

ReLU *Rectified Linear Unit*. vii, 10, 28

ROC *Receiver Operating Characteristics*. vii, 26

SGD *Stochastic Gradient Descent*. 13, 39

TN *True negative*. 25

TP *True positive*. 24, 25

ViT *Vision Transformer*. 29

ViViT *Video Vision Transformer*. 1, 31

Lista de Símbolos

$\sum$	Somatório
ϕ	Função de ativação
∇	Vetor gradiente
ϵ	Função de erro
η	Taxa de aprendizado

Sumário

1 Introdução 1
1.1 Justificativa 2
1.2 Objetivos 2
1.3 Organização do Trabalho 3
2 Referencial Teórico 4
2.1 Fundamentação Teórica 4
2.1.1 Inteligência Artificial 4
2.1.2 Aprendizado de Máquina 5
2.1.3 Redes Neurais 6
2.1.3.1 Perceptron 7
2.1.3.2 Perceptron Multicamadas 8
2.1.4 Funções de ativação 9
2.1.5 Função de erro 11
2.1.6 Aprendizado em profundidade 12
2.1.7 Redes Neurais Convolucionais 15
2.1.8 Redes generativas Adversariais 17
2.1.9 Transferência de aprendizado 19
2.1.10 Aumento de dados 20
2.1.11 Generalização 20
2.1.12 Regularização 22
2.1.12.1 *Dropout* 22
2.1.12.2 Paragem antecipada 23
2.1.13 Otimização de hiper-parâmetros 23
2.1.14 Métricas de avaliação 24
2.1.15 *Deepfake* 26
2.2 Trabalhos Relacionados 27
3 Desenvolvimento 32
3.1 Conjunto de dados 32
3.2 Pré-Processamento dos dados 34
3.3 Modelo proposto 37
4 Resultados 40
4.1 Configuração dos experimentos 40
4.2 Resultados obtidos 40
4.3 Discussão dos resultados 43
5 Considerações Finais 46
5.1 Conclusão 46

5.2 Trabalhos Futuros 46

Referências 48

1 Introdução

Em 2018, o diretor e ator Jordan Peele gravou um vídeo imitando o ex-presidente dos Estados Unidos, Barack Obama. Contudo, o vídeo se inicia somente com a imagem de Obama, possuindo movimentos labiais extremamente convincentes (SILVERMAN, 2018).

Para o processamento desse vídeo, foram necessárias 56 horas de processamento, o que inviabiliza a geração desse tipo de conteúdo para a maioria das pessoas (SILVERMAN, 2018). Entretanto, com o melhoramento da técnica e a criação de *hardwares* mais eficientes, o crescimento da quantidade de vídeos falsos na internet tornava-se iminente.

A esses vídeos gerados por técnicas de inteligência artificial que são extremamente críveis e que alteram o rosto, voz, fala ou características das pessoas, é atribuído o nome *deepfake*. Este consiste em uma mistura entre os termos *deep learning*, a técnica utilizada para a geração das mídias, e *fake*, referindo-se à não autenticidade dos conteúdos gerados (MIRSKY; LEE, 2021).

Entre os anos de 2022 e 2023, a presença de vídeos com a presença de *deepfakes* na internet triplicou, gerando consequências significativas para diferentes indivíduos (ŁABUZ; NEHRING, 2024). Um exemplo notável ocorreu com Muharrem Ince, candidato à presidência da Turquia, que desistiu da corrida presidencial após falsos vídeos sexuais manipulados com a sua face serem compartilhados na internet (KIRBY, 2023; ŁABUZ; NEHRING, 2024).

Em Hong Kong, uma multinacional do setor financeiro transferiu cerca de 25 milhões de dólares para golpistas depois que um dos funcionários da empresa recebeu um e-mail do chefe financeiro da sede e participou de uma videoconferência com ele e outros colaboradores. No entanto, todos os participantes da reunião pareciam e soavam como as pessoas reais, mas eram *deepfakes* (CHEN; MAGRAMO, 2024).

Logo, visando diferenciar conteúdos falsos daqueles verdadeiros, pesquisas sobre métodos de classificação foram feitas, utilizando, principalmente, *Convolutional Neural Network* (CNN), como evidenciado por Heidari et al. (2023). Trabalhos como o de Masud et al. (2023) utilizam CNN pré-treinadas para que aprendam as características do que é real e falso para ajudar na decisão final. Outros estudos propõem hipóteses menos tradicionais, como ocorre em Hernandez-Ortega et al. (2022), que utiliza um modelo base para extrair os batimentos cardíacos com base em aspectos fisiológicos. E, utilizando técnicas mais avançadas, Shahzad et al. (2023) fazem o uso de *Video Vision Transformer* (ViViT) para ajudar na classificação dos vídeos.

Assim, esse trabalho busca responder a seguinte pergunta de pesquisa: É possível criar um modelo para detecção de deepfakes que seja competitivo aos mais avançados, mas que utiliza técnicas que demandem menos poder computacional?

Em resultados de pesquisas preliminares, foi possível a construção de um modelo de

redes neurais baseado na rede convolucional Xception que atingiu uma acurácia de 97,30% e um *Area Underneath the Curve* (AUC) de 0,9957 para a base de dados Celeb-DF v2, utilizando somente 24 quadros dos vídeos da mesma. Assim, tais resultados serão utilizados como base para melhorias na segunda parte desse projeto.

1.1 Justificativa

O avanço acelerado das técnicas de inteligência artificial tem trazido impactos significativos em diversas áreas da sociedade, sendo a geração de *deepfakes* uma das aplicações mais preocupantes. Esses conteúdos sintéticos, que manipulam rostos, expressões e até vozes humanas com alto grau de realismo, têm sido utilizados em fraudes, golpes financeiros e disseminação de desinformação (KIRBY, 2023; ŁABUZ; NEHRING, 2024; CHEN; MAGRAMO, 2024). A crescente acessibilidade a ferramentas de criação de *deepfakes* torna urgente o desenvolvimento de mecanismos eficazes de detecção, especialmente diante de casos emblemáticos que demonstram seu potencial nocivo em contextos políticos e econômicos (HEIDARI et al., 2023; MIRSKY; LEE, 2021).

Nesse cenário, o presente trabalho busca contribuir com a pesquisa na área de segurança digital por meio da proposição e avaliação de modelos computacionalmente eficientes para a identificação automática de vídeos falsos. Ao propor alternativas que combinem desempenho competitivo e baixo custo computacional, o trabalho visa ampliar o acesso a soluções de detecção, promovendo uma internet mais segura e confiável.

1.2 Objetivos

O presente trabalho tem como objetivo principal o desenvolvimento de um modelo de inteligência artificial baseado em redes neurais para a detecção de vídeos manipulados por técnicas de *deepfakes*, que seja competitivo com os trabalhos propostos na literatura ao analisar métricas como acurácia e AUC, mas que seja, também, menos computacionalmente custoso.

Para atingir esse objetivo geral, os seguintes objetivos específicos foram definidos:

- Investigar e comparar abordagens recentes para a detecção de *deepfakes* em vídeos, identificando suas principais características, méritos e limitações;
- Avaliar diferentes técnicas de pré-processamento dos vídeos que irão compor a base de dados utilizada;
- Implementar um modelo de redes neurais capaz de realizar a classificação da presença de manipulações em vídeos

- Avaliar o desempenho do modelo proposto em conjuntos de dados públicos, comparando os resultados com trabalhos relacionados da literatura;
- Analisar a relação entre desempenho do modelo e complexidade computacional, destacando cenários nos quais o modelo proposto apresenta vantagens em custo-benefício;

1.3 Organização do Trabalho

Este trabalho está estruturado de forma a introduzir, inicialmente, os conceitos fundamentais para sua compreensão. O Capítulo 2 apresenta uma revisão bibliográfica que contextualiza os principais temas abordados, como inteligência artificial e *deepfakes*. Em seguida, o Capítulo 3 descreve detalhadamente a metodologia empregada na condução dos experimentos, com foco no alcance dos objetivos geral e específicos. Os resultados obtidos são apresentados e discutidos no Capítulo 4. Por fim, o Capítulo 5 reúne as considerações finais do estudo e propõe direções para trabalhos futuros.

2 Referencial Teórico

A Seção 2.1 deste capítulo busca apresentar e desenvolver os principais conceitos necessários para o entendimento da pesquisa desenvolvida. Já a Seção 2.2 apresenta estudos prévios que abordam o mesmo tema e apresentam diferentes características, servindo de base para a formação de um conhecimento inicial sobre as abordagens desenvolvidas, além de ser útil para a comparação dos resultados obtidos.

2.1 Fundamentação Teórica

Para compreender as técnicas de detecção de *deepfakes*, é fundamental, antes, entender o que eles são e como são gerados. Para isso, esta seção apresenta uma base teórica que contempla os principais conceitos envolvidos: começando pela inteligência artificial e sua capacidade de simular comportamentos inteligentes, passando pelo aprendizado de máquina, que permite aos sistemas extrair padrões a partir de dados, e culminando com as redes neurais, abordando desde os fundamentos até arquiteturas mais avançadas, como redes convolucionais e generativas adversariais. Com essa fundamentação consolidada, torna-se possível compreender de forma aprofundada o funcionamento e os desafios inerentes à geração e à detecção de conteúdos manipulados por meio de técnicas de *deepfake*.

2.1.1 Inteligência Artificial

O termo inteligência artificial não possui uma definição única e formal, mas é compreendido como o campo de estudo voltado para o desenvolvimento de sistemas computacionais capazes de realizarem tarefas que, para os seres humanos, exigiriam algum grau de inteligência (RUSSELL; NORVIG, 2021).

Para a execução dessas tarefas, tais sistemas contam com os chamados agentes, entidades que percebem o mundo ao seu redor e são capazes de tomarem decisões com base nas informações que obtêm a partir de sensores como radares, sonares, etc. (RUSSELL; NORVIG, 2021).

Inicialmente, os primeiros avanços da inteligência artificial ocorreram na solução de problemas considerados intelectualmente complexos para seres humanos, mas que podiam ser formalizados de maneira matemática. Por outro lado, problemas aparentemente simples e intuitivos para nós se mostraram extremamente desafiadores para sistemas computacionais, uma vez que podem necessitar de conhecimentos externos e contextuais aprendidos no cotidiano. Esse tipo de conhecimento pode ser extremamente complexo de ser formalizado ou incorporado em um sistema computacional, como exemplos temos um sistema que reconhece palavras faladas, mesmo com um sotaque (GOODFELLOW; BENGIO; COURVILLE, 2016).

Em 1959, Arthur L. Samuel propôs um agente capaz de jogar damas enquanto aprendia com suas jogadas a fim de melhorar e vencer jogos (SAMUEL, 1959). Já em 1997, o supercomputador da IBM, o Deep Blue II, derrotou Garry Kasparov, na época, atual campeão mundial de xadrez, com uma busca em árvore e a utilização de créditos para a seleção das melhores jogadas (CAMPBELL; HOANE; HSU, 2002).

Atualmente, a inteligência artificial está presente em diferentes áreas do cotidiano, seja com reconhecimento de faces para a identificação de pessoas em fotos no Facebook, para a condução de veículos autônomos, reconhecimento de voz para assistentes virtuais (KAPLAN; HAENLEIN, 2019), na geração de textos com o ChatGPT (RADFORD et al., 2018) e outras diversas aplicações.

2.1.2 Aprendizado de Máquina

Os agentes de inteligência artificial necessitam de informações sobre o ambiente em que estão inseridos. Para isso, muitos dados são inseridos de forma formal, a fim de permitir que o computador consiga raciocinar em uma estratégia conhecida como base de conhecimento (GOODFELLOW; BENGIO; COURVILLE, 2016).

No entanto, um dos maiores desafios para essa abordagem encontra-se na dificuldade da formulação das descrições do ambiente de forma precisa. Essa limitação foi superada com o avanço de técnicas que permitem que a própria inteligência artificial receba dados e extraia informações relevantes para o problema, em uma tarefa chamada de aprendizado de máquina (GOODFELLOW; BENGIO; COURVILLE, 2016).

O processo de aprendizado de máquina ocorre quando um programa de computador aprende, a partir de uma experiência, para um conjunto de atividades específicas, padrões que otimizam a sua performance nas tarefas das quais ele foi proposto a realizar (MITCHELL; MUNSON, 1997).

O aprendizado de máquina pode ser dividido em quatro grandes campos: aprendizado supervisionado, semi-supervisionado, não supervisionado e por reforço. Segundo Burkov (2019), eles podem ser definidos da seguinte forma:

- No **aprendizado supervisionado**, dado um conjunto de dados de entrada X, o objetivo é treinar um algoritmo capaz de aprender a função que melhor relaciona esses dados com os seus respectivos rótulos presentes no conjunto Y, uma vez que há a correspondência de um-para-um, ou seja, um dado de entrada refere-se a uma única saída. Em outras palavras, seria como um aluno tentando realizar uma tarefa enquanto um professor aponta onde está correto ou errado.

- No **aprendizado semi-supervisionado**, o conjunto de dados é dividido em duas partes: os dados rotulados, que funcionam como guia direto do modelo, fornecendo a associação

entre entradas e saídas esperadas de forma semelhante ao aprendizado supervisionado, e os dados não rotulados, que não possuem rótulos, mas são aproveitados para revelar padrões, estruturas e distribuições subjacentes no espaço de entrada. Dessa forma, enquanto os dados rotulados oferecem a base de referência, os dados não rotulados ampliam o conhecimento sobre o problema, permitindo ao modelo generalizar melhor e produzir predições mais robustas e precisas. É como se um aluno aprendesse a resolver equações matemáticas a partir de exemplos do professor e, depois, aplicasse o que entendeu para solucionar problemas novos.

- No **aprendizado não supervisionado**, apenas os dados de entrada são conhecidos. Nesse caso, busca-se encontrar uma função que transforme os dados ou revele estruturas ou padrões úteis, como agrupamentos ou distribuições latentes. Tal tipo de aprendizado se assemelha a de um entusiasta em programação que decide aprender tudo sem a instrução de terceiros.

- No **aprendizado por reforço**, o agente interage com o ambiente em que está inserido, realizando diferentes ações que resultam em recompensas ou penalidades. Com base nesse *feedback*, o agente aprende a tomar decisões que maximizem suas recompensas ao longo do tempo. No mundo real, tal comportamento pode ser visto na criação de crianças, que recebem palavras de afirmação para comportamentos corretos, mas são corrigidas quando realizam algo errado.

2.1.3 Redes Neurais

Para o bom funcionamento dos algoritmos de aprendizado de máquina, é fundamental que os dados passem inicialmente por um processo de extração de características e identificação de padrões relevantes. No entanto, essa tarefa costuma ser complexa e demorada, exigindo a atuação de especialistas em domínios específicos, o que eleva significativamente o custo de desenvolvimento. Como consequência, esse processo dificulta a generalização dos algoritmos e faz com que muitos deles sejam projetados para resolver apenas problemas muito específicos (GOODFELLOW; BENGIO; COURVILLE, 2016; LECUN; BENGIO; HINTON, 2015).

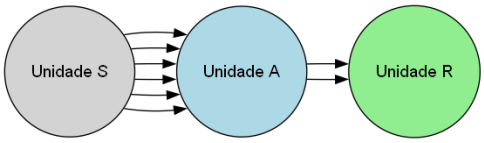
A automatização da tarefa de encontrar boas representações para os dados pode ser feita com um algoritmo de aprendizado para uso geral, sendo essa a característica principal do aprendizado em profundidade, o *deep learning* (LECUN; BENGIO; HINTON, 2015). Em seu funcionamento, ao invés de tentar representar a informação como um todo, suas características são quebradas em partes menores para, no final, entender o todo (GOODFELLOW; BENGIO; COURVILLE, 2016).

2.1.3.1 Perceptron

Para realizar a representação através de pequenas informações, Rosenblatt (1958) explora a teoria de que o aprendizado em seres vivos se dá por modificações nas conexões entre os neurônios do cérebro.

Rosenblatt (1958) propõe a simulação de um sistema neural baseado em conexões, o perceptron, um sistema composto por três camadas: unidades S, que detectam estímulos sensoriais, como os olhos de um animal que vê um predador, unidades A, que recebem os estímulos e os processam de forma não linear, como o cérebro do animal que identificará o que foi visto, processar e decidir qual ação tomar, e as saídas do sistema, as unidades R, que, no exemplo do animal, seria a decisão de sair o mais rapidamente possível daquela região. Assim, a representação de todo esse sistema pode ser observado na Figura 2.1

Figura 2.1 – Adaptação do perceptron de (ROSENBLATT, 1958).



Fonte: Imagem do autor

Os dados da unidade S vão para a unidade A, onde cada um deles sofrerá uma modificação com base em um peso, como pode ser observado na formulação matemática:

$$a_j = \sum_{i=1}^{N} w_{ij} * s_i, \tag{2.1}$$

onde:

- a_j: Valor da unidade de ativação j;
- N: número total de unidades S conectadas à unidade A;
- $w_{ij} \in \mathbb{R}$: peso da conexão entre s_i e a_j;
- s_i: Valor da i-ésima unidade sensorial.

Sendo que, a unidade A somente será considerada como ativada quando o valor de a_j for, pelo menos, o limiar de ativação θ, ou seja: $a_j \geq \theta$. Dessa forma, é feita uma função de ativação conhecida como degrau, que resulta somente em valores 0 e 1

Por fim, a saída final do sistema é realizada por meio de uma soma ponderada entre esses valores e os pesos da unidade R.

2.1.3.2 Perceptron Multicamadas

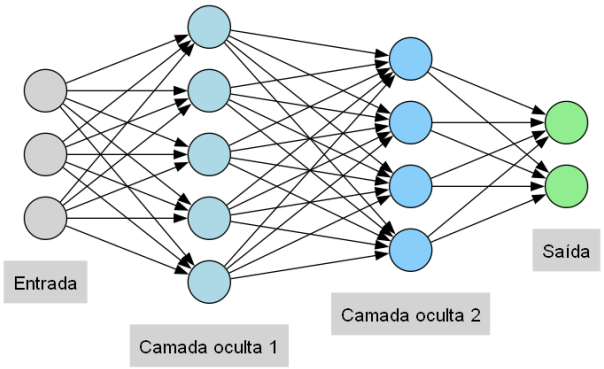
Mesmo demonstrando o poder de aprendizado do perceptron, Rosenblatt (1958) discute sobre como ele falha ao lidar com abstrações mais complexas, necessitando a formulação de um sistema mais avançado.

Tal sistema pode ser feito a partir do aumento da quantidade de perceptrons, comumente chamados de neurônios, e separando-os em diferentes camadas, onde um perceptron da camada L está conectado com todos os outros da camada $L + 1$, gerando um perceptron multicamadas, ou em inglês *Multilayer perceptron* (MLP) (GOODFELLOW; BENGIO; COURVILLE, 2016; ZHANG et al., 2023).

Entre a camada de entrada e a de saída, são adicionadas quantas camadas intermediárias forem consideradas necessárias, sendo conhecidas como camadas ocultas, uma vez que elas não são diretamente observadas, sendo esta uma propriedade somente da camada de entrada e da de saída, caracterizando a aprendizagem em profundidade (GOODFELLOW; BENGIO; COURVILLE, 2016; ZHANG et al., 2023).

A Figura 2.2 exibe um exemplo de um MLP com três neurônios na camada de entrada, cinco neurônios na primeira camada oculta, quatro na segunda e dois na camada de saída.

Figura 2.2 – MLP com uma camada de entrada, duas ocultas e uma de saída



Fonte: Imagem do autor

A informação em um MLP é propagada de maneira sequencial, ou seja, a partir da camada L, a informação vai para a $L + 1$ e não retorna para a anterior (GOODFELLOW; BENGIO; COURVILLE, 2016).

Os pesos dos neurônios de uma camada podem ser definidos de forma matricial como w^l_{jk}, onde l é a camada em questão, k é o neurônio da camada anterior e j é o neurônio da camada l (NIELSEN, 2015).

Um viés (também conhecido como *bias*) pode ser adicionado à rede para ter a funcionalidade de um peso a mais. O seu valor é semelhante ao limiar θ do perceptron, mas com o valor oposto. Cada neurônio da camada possui o seu próprio *bias*, e ele pode ser representado como b^l_j, onde l indica a camada atual e j o neurônio em questão (NIELSEN, 2015).

Dessa forma, temos a soma ponderada para a saída de um neurônio com respeito a todos os seus valores de entrada, pesos e bias (NIELSEN, 2015), como pode ser visto a seguir:

$$z_j^l = \sum_j w_{jk}^l * x_k^{l-1} + b_j^l, \tag{2.2}$$

onde:

- z_j^l refere-se ao valor de saída do neurônio j da camada l;
- x_k^{l-1} refere-se ao valor de saída da camada anterior ou ao valor de entrada do modelo.

Seguindo tal representação, a rede neural seria uma composição linear de diferentes funções. Dessa forma, para adicionar uma não linearidade nos dados, o valor de saída z^l é modificado por uma função de ativação ϕ não linear (ZHANG et al., 2023). Dessa forma, a saída de um neurônio passa a seguinte equação:

$$a_j^l = \phi(z_j^l). \tag{2.3}$$

Sendo X o conjunto de dados de entrada, W^l a representação matricial de todos os pesos de uma camada, B^l os *bias* da camada l referentes a seus respectivos neurônios, então a Equação 2.4 refere-se à formulação matemática de uma rede neural com duas camadas. Ela pode ser definida por:

$$Z = \phi(W^1(\phi(W^0 X + B^0)) + B^1). \tag{2.4}$$

Dessa forma, fica evidente que os valores de um neurônio terão influência no resultado final do processo de propagação dos dados.

2.1.4 Funções de ativação

Como dito anteriormente, os modelos necessitam de uma função de ativação não linear para evitar que uma rede neural seja somente uma composição linear entre as entradas e saídas, uma vez que uma composição linear é apenas uma função linear que pode ser simplificada.

Ademais, uma característica essencial para as funções de ativação é o fato de serem diferenciáveis, uma vez que o algoritmo de otimização utilizará a derivada das mesmas para encontrar o novo valor para os parâmetros do modelo (ZHANG et al., 2023).

Dessa forma, algumas das principais funções de ativação serão apresentadas a seguir, sendo que, diferentemente do perceptron visto na Seção 2.1.3.1, podem resultar em valores diferentes do que somente 0 e 1.

Sigmoid

A função *sigmoid* tem a característica de "espremer" diferentes valores para o intervalo $[0, 1]$, sendo muito utilizada para uma classificação binária (ZHANG et al., 2023). Ela pode ser definida por:

$$sigmoid(x) = \frac{1}{1 + exp(-x)}. \tag{2.5}$$

Já a sua derivada pode ser representada pela equação:

$$\frac{d}{dx} sigmoid(x) = sigmoid(x)(1 - sigmoid(x)). \tag{2.6}$$

Rectified Linear Unit

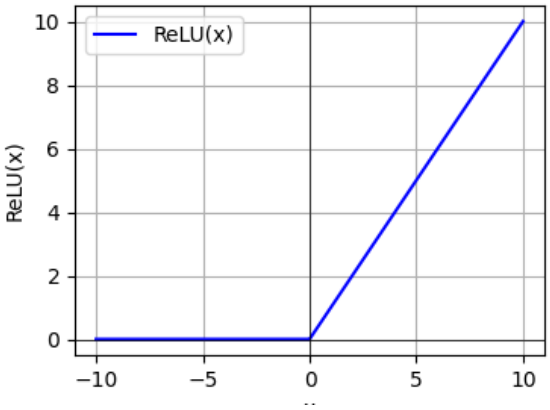
Uma das mais famosas e utilizadas funções de ativação é a Unidade Linear Retificada, ou em inglês *Rectified Linear Unit* (ReLU), onde somente informações positivas são mantidas e qualquer outra é descartada (ZHANG et al., 2023). Ela pode ser definida matematicamente por:

$$\text{ReLU}(x) = \begin{cases} 0 & \text{, se } x < 0 \\ x & \text{, se } x \geq 0 \end{cases}. \tag{2.7}$$

Observando o gráfico da sua função na Figura 2.3, observa-se um ponto de "quina" quando $x = 0)$, impedindo a diferenciação da função. Entretanto, para evitar tal problema, sua derivada é definida pela equação:

$$\frac{d}{dx}\text{ReLU}(x) = \begin{cases} 0 & \text{, se } x < 0 \\ 1 & \text{, caso contrário} \end{cases} \tag{2.8}$$

Figura 2.3 – Função ReLU



Fonte: Imagem do autor

Softmax

Para problemas de classificação com múltiplas classes, uma forma comum de representar os rótulos é por meio de vetores *one-hot*, ou seja, vetores de N posições (onde N é o número de classes), com todos os valores iguais a zero, exceto na posição correspondente à classe correta, que recebe o valor 1.

Para que a saída de uma rede neural possa ser interpretada como uma distribuição de probabilidade sobre essas classes, é necessário que ela contenha N neurônios, e que seus valores resultem em números não-negativos cuja soma total seja igual a 1. No entanto, as saídas brutas da rede nem sempre satisfazem essas propriedades, podendo conter valores negativos e somas arbitrárias (ZHANG et al., 2023).

Para resolver esse problema, utiliza-se a função *softmax*, que transforma os *scores* produzidos pela rede em uma distribuição de probabilidade válida, garantindo que todos os valores estejam no intervalo $[0, 1]$ e que a soma total seja 1 (ZHANG et al., 2023).

A função *softmax* é definida matematicamente por:

$$softmax(x_i) = \frac{e^{x_i}}{\sum_{j=1}^{K} e^{x_j}}, \tag{2.9}$$

onde:

- x_i representa o escore atribuído pelo modelo à classe i;
- K é o número total de classes no problema;
- $\sum_{j=1}^{K} e^{x_j}$ é o somatório das exponenciais de todos os valores atribuídos pelo modelo, servindo como fator de normalização.

2.1.5 Função de erro

Para avaliar a eficiência de um modelo de rede neural supervisionado, verifica-se a discrepância entre os valores previstos e os valores reais esperados (ELHARROUSS et al., 2025).

Cada tipo de tarefa impõe requisitos específicos sobre o modelo. Em tarefas de regressão, a saída esperada é um valor contínuo; já em classificação, espera-se um rótulo discreto. Assim, a escolha da função de erro (ou função de perda) deve ser coerente com a natureza da tarefa. Não é apropriado, por exemplo, empregar uma função de perda voltada à classificação em tarefas generativas, pois, enquanto a primeira visa maximizar a taxa de acertos, a segunda busca produzir saídas com aparência realista (ELHARROUSS et al., 2025).

O cálculo da função de perda considera dois elementos principais: o valor real ou esperado, representado por y, e o valor estimado pelo modelo, denotado por $\hat{y}$.

A seguir a definição dos principais tipos de erro utilizados em aprendizado de máquina:

Erro Médio Absoluto

Em tarefas de regressão, o Erro Médio Absoluto, ou em inglês *Mean Absolute Error* (MAE), indica o quão distante o valor previsto ficou do valor original na mesma unidade de medida, gerando uma média para todos os N valores calculados (ELHARROUSS et al., 2025). Ele é definido por:

$$L(y, \hat{y}) = \frac{1}{N} \sum_{i=1}^{N} |y_i - \hat{y}_i|. \tag{2.10}$$

Erro Médio Quadrático

Para tarefas de regressão, o Erro Quadrático Médio, ou em inglês *Mean Absolute Error* (MSE), penaliza medidas distantes ao utilizar o quadrado da distância entre os valores previstos e esperados, ou seja, erros crassos sofrerão uma punição maior do que erros pequenos (ELHARROUSS et al., 2025). O MSE é definido por:

$$L(y, \hat{y}) = \frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2. \tag{2.11}$$

Erro de Entropia Cruzada

Para tarefas de classificação, o Erro de Entropia Cruzada (*Cross-Entropy Loss*) avalia a diferença entre as distribuições de probabilidade para cada uma das classes do problema, onde os valores previstos somente podem variar entre 0 e 1 (ELHARROUSS et al., 2025). Ele é definido por:

$$L(y, \hat{y}) = -\sum_{i=1}^{N} y_i log(\hat{y}_i). \tag{2.12}$$

Erro de Entropia Cruzada Binária

Para tarefas de classificação com somente duas classes, o Erro de Entropia Cruzada Binária, ou em inglês *Binary Cross-Entropy Loss* (BCE), calcula a distância entre os valores binários previstos e esperados (ELHARROUSS et al., 2025). O BCE é definido por:

$$L(y, \hat{y}) = -(ylog(\hat{y}) + (1 - y)log(1 - \hat{y})). \tag{2.13}$$

2.1.6 Aprendizado em profundidade

Após o resultado de uma predição e com o seu devido erro calculado, pode-se, então, iniciar o processo para a atualização dos parâmetros do modelo, ou seja, modificar os pesos e o *bias* para que a distância entre o valor predito e o valor esperado seja o mais próxima de zero quanto possível (LECUN; BENGIO; HINTON, 2015).

Para a realização do aprendizado, uma técnica muito comum é a aplicação do algoritmo de descida do gradiente. Onde, dada uma função com múltiplos parâmetros, o seu objetivo é

encontrar a combinação que resultará no menor valor possível da função, ou seja, o mínimo local (ALAGöZLü, 2022).

Em um aprendizado supervisionado de uma rede neural, após uma predição, o erro é calculado. Esse valor é uma variável dependente de todos os pesos e *bias* da rede, e encontra-se em algum local no plano multi-dimensional do modelo para os parâmetros atuais. A fim de encontrar a direção em que a função deverá decrementar o seu valor, o vetor gradiente do erro em função dos pesos pode ser calculado (LECUN; BENGIO; HINTON, 2015).

O vetor gradiente de uma função é a direção na qual os valores da função terão, naquele determinado ponto, a maior taxa de crescimento (STEWART, 2017). Contudo, como o objetivo é encontrar um mínimo local, e não um máximo local, então o oposto deste vetor deve ser utilizado (LECUN; BENGIO; HINTON, 2015).

A fim de garantir uma descida suave, o vetor gradiente é multiplicado por um escalar conhecido como taxa de aprendizado (*learning rate*), representado por η. O seu valor define o tamanho do passo para atingir o mínimo local, onde um valor muito pequeno demandará mais tempo, enquanto um valor muito grande pode acelerar o processo, mas pode impedir que o mínimo local seja atingido, fazendo com que a função fique constantemente oscilando próximo àquele ponto (ALAGöZLü, 2022).

Com o erro calculado a partir de todos os dados do conjunto de dados, pode ser dado um passo na atualização dos pesos e *bias* a partir da fórmula geral da descida do gradiente, definida pela equação:

$$\theta = \theta - \eta\nabla(E), \tag{2.14}$$

onde:

- θ refere-se aos pesos ou *bias*
- η refere-se ao *learning rate*
- $\nabla(E)$ refere-se ao vetor gradiente dos pesos ou *bias* em relação ao erro da função

Para que o algoritmo não necessite realizar a predição e cálculo do erro de todos os dados para que só então seja feito um passo na descida do gradiente, é possível realizar um passo a cada predição. Tal processo é conhecido como a Descida de Gradiente Estocástica, ou em inglês *Stochastic Gradient Descent* (SGD), e acelera o processo de convergência dos parâmetros, uma vez que ele está sempre se atualizando, mesmo que de pouco em pouco (LECUN; BENGIO; HINTON, 2015; ALAGöZLü, 2022).

Utilizando ambas as abordagens em um conjunto de dados, o conjunto de dados pode ser dividido em diversos lotes pequenos, conhecidos como mini-lotes, ou em inglês *mini-batch*. Assim, para cada mini-lote, são feitas suas predições e cálculo dos erros, que serão agregados e utilizados para o passo de descida do gradiente e atualização dos parâmetros, permitindo

que, na mesma iteração de dados, avanços sejam feitos de forma procedural e que o próximo mini-lote contribua um pouco mais com o avanço que foi beneficiado. Dessa forma, evita-se ficar constantemente atualizando a rede, que é um processo custoso, ao mesmo tempo que não precisará esperar pela predição de toda a base para realizar um pequeno passo (ALAGöZLü, 2022).

Para a computação dos vetores gradientes dos pesos e *bias*, é necessário um algoritmo eficiente, uma vez que as redes neurais podem possuir uma grande quantidade de parâmetros a serem ajustados. Dessa forma, foi proposto o algoritmo de retropropagação, *backpropagation* em inglês, em Rumelhart, Hinton e Williams (1986).

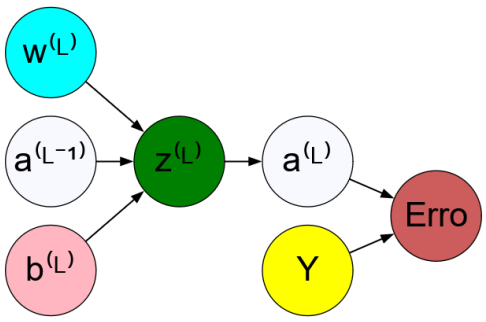
O algoritmo tem o objetivo de calcular o vetor gradiente para cada peso e *bias*, indicando como pequenas variações em seus valores podem modificar o erro final, que é o objetivo a ser minimizado.

A Figura 2.4 exibe um grafo de uma rede neural abstrata com alguns passos e itens necessários para calcular o erro de uma predição, que pode ser realizado pela equação:

$$Erro = \varepsilon(\phi(w^l * a^{l-1} + b^l), Y), \tag{2.15}$$

onde ε refere-se a uma função de erro arbitrária.

Figura 2.4 – Representação da computação do erro de uma rede neural



Fonte: *Imagem do autor.*

Para calcular o quanto as modificações w^L impactam o erro, será necessário o seu vetor gradiente em relação ao erro. Todavia, tal cálculo não pode ser feito de forma direta, uma vez que mudanças em w^L geram modificações em z^L, depois em a^L e, por fim, no erro.

Dessa forma, para circundar tal problema, é necessário aplicar a regra da cadeia para identificar o quanto cada mudança impactará no resultado final. Portanto, seguindo o grafo da Figura 2.4 em seu sentido oposto, é possível montar uma equação que relaciona cada item com aquele que possui uma interferência direta, como pode ser observado pela equação:

$$\frac{\partial \varepsilon}{\partial w^L} = \frac{\partial \varepsilon}{\partial a^L} \frac{\partial a^L}{\partial z^L} \frac{\partial z^L}{\partial w^L}. \tag{2.16}$$

O cálculo do vetor gradiente do *bias* é semelhante ao dos pesos, como pode ser visto na seguinte equação:

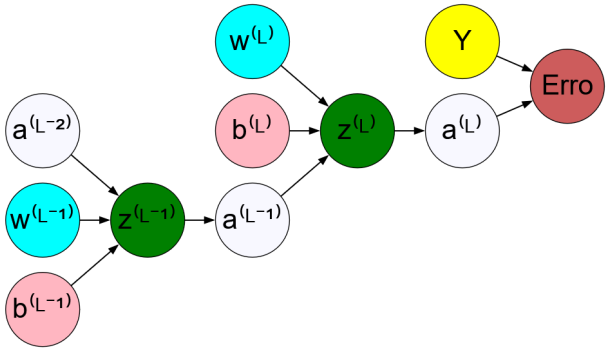
$$\frac{\partial \varepsilon}{\partial b^L} = \frac{\partial \varepsilon}{\partial a^L} \frac{\partial a^L}{\partial z^L} \frac{\partial z^L}{\partial b^L}. \tag{2.17}$$

Para a presença de mais camadas, como na Figura 2.5, o processo é similar. Ou seja, basta ir caminhando no grafo seguindo o sentido oposto e adicionando na função os termos diretamente relacionados. Assim, a Equação 2.18 refere-se a w^{L-1}, enquanto a Equação 2.19 refere-se a b^{L-1}, se aproveitando dos cálculos já realizados das derivadas parciais, evitando operações matemáticas repetitivas e garantindo uma execução viável do algoritmo.

$$\frac{\partial \varepsilon}{\partial w^{L-1}} = \frac{\partial \varepsilon}{\partial a^L} \frac{\partial a^L}{\partial z^L} \frac{\partial z^L}{\partial a^{L-1}} \frac{\partial a^{L-1}}{\partial z^{L-1}} \frac{\partial z^{L-1}}{\partial w^{L-1}}, \tag{2.18}$$

$$\frac{\partial \varepsilon}{\partial b^{L-1}} = \frac{\partial \varepsilon}{\partial a^L} \frac{\partial a^L}{\partial z^L} \frac{\partial z^L}{\partial a^{L-1}} \frac{\partial a^{L-1}}{\partial z^{L-1}} \frac{\partial z^{L-1}}{\partial b^{L-1}}. \tag{2.19}$$

Figura 2.5 – Representação da computação do erro de uma rede neural



Fonte: *Imagem do autor*.

2.1.7 Redes Neurais Convolucionais

Como visto em Gonzalez e Woods (2018), uma imagem em preto e branco, ou escala de cinza, pode ser entendida como uma função $f(x, y)$ de coordenadas espaciais em um plano, onde x indica a linha e y uma coluna. Dessa forma, $f(x, y)$ representa a intensidade do pixel na imagem, variando de 0 (preto) a 255 (branco), para imagens de 8-bit.

Por outro lado, imagens coloridas são representadas como uma matriz tridimensional, ou uma função $f(x, y, z)$, onde z indica qual o canal de cor, podendo ser vermelho, verde ou azul, formando o espectro de cores RGB.

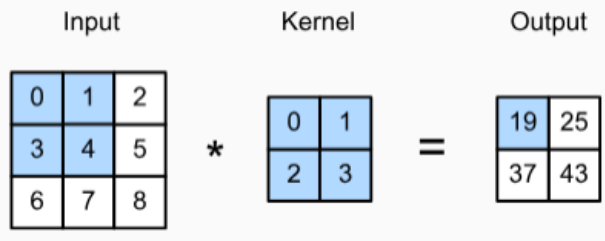
Em 2012, durante o Desafio de Reconhecimento Visual em Larga Escala ImageNet (*ImageNet Large-Scale Visual Recognition Challenge*), com o objetivo de classificar corretamente mais de 1,2 milhões de imagens em 1.000 classes diferentes, um grupo de pesquisadores optaram por não utilizar técnicas clássicas de extração de características, mas sim por utilizar Redes

Neurais Convolucionais, ou em inglês *Convolutional Neural Network* (CNN). Dessa forma, tal escolha rendeu o melhor resultado da competição com o erro médio sendo a metade do competidor que ficou em segundo lugar (KRIZHEVSKY; SUTSKEVER; HINTON, 2012).

Em uma CNN, os pesos da rede neural estarão localizados em uma estrutura conhecida como filtro (também chamado de *kernel*), de tamanho menor do que a imagem e se deslocará sobre ela extraindo características dos pixeis vizinhos (NIELSEN, 2015).

A Figura 2.6 mostra o processo de convolução em uma matriz 3×3 por um filtro 2×2, começando pelo canto superior esquerdo da matriz e deslocando uma unidade para a direita até o fim da linha. Ao chegar no final, o filtro volta para a primeira coluna e ele é reposicionado para a linha de baixo (ZHANG et al., 2023).

Figura 2.6 – Operação de convolução



Fonte: (ZHANG et al., 2023)

Como mostrado por Nielsen (2015), o processo de convolução para cada posição do filtro pode ser entendido pela equação:

$$\phi(b + \sum_{l=0}^{n} \sum_{m=0}^{n} w_{l,m} a_{i+l,j+m}), \tag{2.20}$$

onde:

- n é o tamanho do filtro
- $w_{l,m}$ é o m-ésimo peso do filtro na linha l
- $a_{i+m,j+m}$ é o pixel da linha $i + l$ e coluna $j + m$ que estão sendo verificados no momento pelo filtro.
- b é o valor do *bias*
- ϕ é a função de ativação

Dessa forma, aplicando a Equação 2.20 para os valores em azul da Figura 2.6, temos o resultado:

$$0 \times 0 + 1 \times 1 + 3 \times 2 + 4 \times 3 = 19, \tag{2.21}$$

onde todos os *bias* foram considerados como zero e a função de ativação uma função identidade.

Ademais, é possível notar que a matriz resultante possui um tamanho menor do que a original, uma vez que somente é possível realizar os cálculos em um espaço da imagem em que o filtro se encaixa totalmente (ZHANG et al., 2023). Assim, o tamanho da matriz resultante de uma convolução pode ser calculado por:

$$(n_h - k_h + 1) \times (n_w - k_w + 1), \tag{2.22}$$

onde:

- n_h é o número de linhas da matriz
- k_h é o número de linhas do filtro
- n_w é o número de colunas da matriz
- k_w é o número de colunas do filtro

Para evitar que ocorresse a diminuição do tamanho, poderia ser aplicado um *padding*, ou seja, acrescentar zeros ao redor da imagem para aumentar o seu tamanho artificialmente (GOODFELLOW; BENGIO; COURVILLE, 2016).

Em muitos casos, pode ser necessário mais de um filtro para poder extrair diferentes características de um alvo, podendo ser adicionados tantos quanto forem desejados. Dessa forma, cada um dos filtros funcionará de maneira semelhante, mas cada um terá o seu próprio conjunto de pesos e *bias*. Por fim, a saída da camada de convolução terá o tamanho definido pela Equação 2.22 e o número de canais igual ao número de filtros previamente definidos (NIELSEN, 2015).

A Figura 2.7 exemplifica a necessidade de mais filtros, onde é possível notar que cada um deles está extraindo diferentes características da imagem de um gato, por mais que em algumas o resultado possa parecer semelhante.

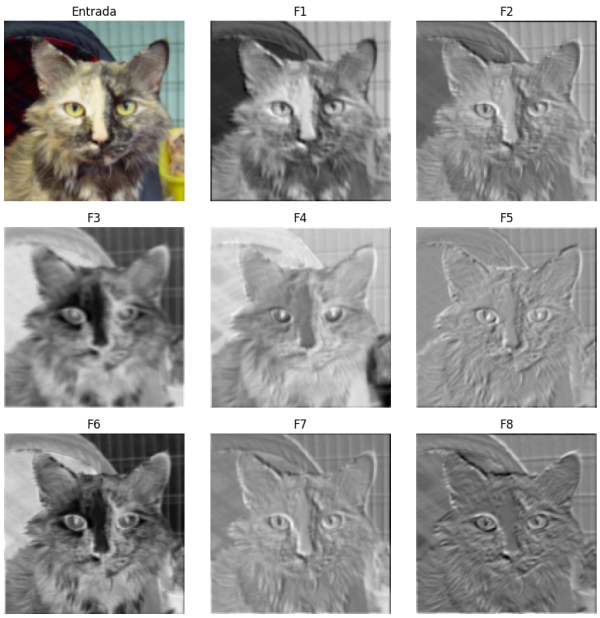
Para lidar com imagens coloridas, ou com o resultado das camadas de convoluções anteriores, o processo será semelhante, mas cada filtro definido deverá possuir a mesma quantidade de camadas que os dados de entrada, onde cada um agirá de forma independente sobre o canal em que está localizado (ZHANG et al., 2023).

A Figura 2.8 exemplifica esse processo. A entrada possui duas camadas, consequentemente, o único filtro também possuirá duas camadas. As operações de convolução são feitas para cada filtro com respeito à sua camada específica e, no final, elas são somadas. Como há somente um filtro, logo a saída terá somente um canal.

2.1.8 Redes generativas Adversariais

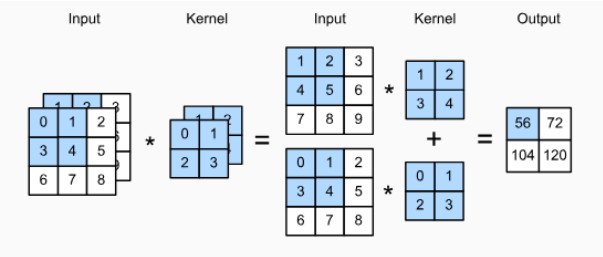
A criação de dados sintéticos para o treinamento de um classificador de imagens pode ajudar quando a quantidade de dados para treino é baixa. Dessa forma, com uma Rede Generativa

Figura 2.7 – Diferentes filtros convolucionais aprendidos e aplicados para a detecção de um gato



Fonte: Adaptado de (JIKADARA, 2023)

Figura 2.8 – Operação de convolução com múltiplas camadas de entrada



Fonte: (ZHANG et al., 2023)

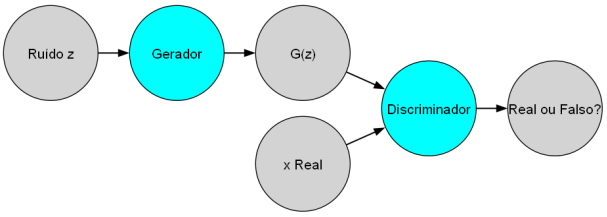
Adversarial, ou em inglês *Generative Adversarial Networks* (GAN), é possível gerar uma imagem altamente realista que se pareça com uma original (ZHANG et al., 2023).

Uma GAN possui um modelo gerador que produzirá as imagens falsas a fim de enganar o modelo discriminador, que tentará prever se a imagem é verdadeira ou não. O objetivo do gerador é ir aprendendo formas mais eficazes de produzir uma saída convincente, enquanto o objetivo do discriminador é aprender os novos padrões para que não seja enganado (GOODFELLOW et al., 2014).

O *pipeline* do funcionamento de uma GAN pode ser entendido pelo diagrama da Figura 2.9. A partir de um ruído de entrada z, o gerador cria um dado falso representado por $G(z)$. Assim, o discriminador recebe um dado de entrada, podendo ser o falso ou um verdadeiro, e decide a veracidade da informação (ZHANG et al., 2023).

O modelo discriminador é feito para gerar uma saída binária, predizendo se o dado é real ou falso. Dessa forma, há o objetivo de minimizar sua função de erro, por exemplo a BCE

Figura 2.9 – Fluxograma de uma Rede Neural Adversarial



Fonte: Adaptado de (ZHANG et al., 2023)

(ZHANG et al., 2023).

O gerador está tentando enganar o discriminador, ou seja, fazer com que a predição da veracidade das informações possua o maior erro possível, logo ele tenta maximizar a função de perda do discriminador (ZHANG et al., 2023).

Dessa forma, a partir dessa tentativa de minimização e maximização do erro, surge a equação a seguir para descrever os resultados dessas funções (ZHANG et al., 2023):

$$\min_G \max_D \left\{ -\mathbb{E}_{\mathbf{x}\sim\text{Data}} \left[\log D(\mathbf{x})\right] - \mathbb{E}_{\mathbf{z}\sim\text{Noise}} \left[\log\left(1 - D(G(\mathbf{z}))\right)\right] \right\}, \tag{2.23}$$

onde:

- $\mathbb{E}_{\mathbf{x}\sim\text{Data}} \left[\log D(\mathbf{x})\right]$ é o objetivo do discriminador ao tentar minimizar a função.
- $\mathbb{E}_{\mathbf{z}\sim\text{Noise}} \left[\log\left(1 - D(G(\mathbf{z}))\right)\right]$ é o objetivo do gerador de criar um dado indistinguível de um real

2.1.9 Transferência de aprendizado

No Japão, a empresa *Brain*, especializada em visão computacional, desenvolveu um sistema chamado *BakeryScan*, capaz de identificar automaticamente diferentes tipos de pães para facilitar a precificação em padarias. Em 2017, ao assistir a uma reportagem na televisão sobre essa tecnologia, um médico do Centro Médico de Pesquisa Louis Pasteur notou que as células cancerígenas vistas ao microscópio apresentavam formatos semelhantes aos dos pães reconhecidos pelo sistema (SOMERS, 2021).

O médico entrou em contato com a *Brain*, que então adaptou o *BakeryScan* para a detecção de patologias em imagens médicas. Posteriormente, o mesmo algoritmo foi modificado para resolver outros problemas de classificação visual. Para isso, o conhecimento previamente adquirido foi reutilizado e ajustado aos novos contextos, acelerando e otimizando o processo de desenvolvimento (SOMERS, 2021).

Embora o *BakeryScan* original não utilize redes neurais, seu processo de adaptação é análogo a uma técnica amplamente empregada no campo do aprendizado de máquina: a transferência de aprendizado. Essa técnica consiste em aproveitar os parâmetros aprendidos por

um modelo em uma tarefa A e reutilizá-los em um novo modelo voltado para uma tarefa B semelhante (SOMERS, 2021; VRBANčIč; PODGORELEC, 2020).

A transferência de aprendizado é especialmente comum em CNN. Por exemplo, uma rede treinada para classificar animais pode ser adaptada para localizá-los em uma imagem, reutilizando as características visuais extraídas anteriormente e ignorando as camadas finais de classificação (VRBANčIč; PODGORELEC, 2020).

Além disso, levando em conta as especificidades de cada conjunto de dados ou desafio, é possível refinar esse modelo pré-treinado por meio de uma técnica chamada *fine-tuning* (ou ajuste fino). Esse processo consiste em adaptar os pesos do modelo de forma mais precisa ao novo domínio, garantindo melhores resultados em tarefas especializadas (VRBANčIč; PODGORELEC, 2020).

2.1.10 Aumento de dados

Embora a transferência de aprendizado diminua a dificuldade para se treinar um modelo, ainda há a necessidade de uma grande quantidade de dados de entrada para que ocorra a devida adequação à nova tarefa, uma vez que o uso de pequenas quantidades de dados podem levar a um super ajuste do modelo aos dados de treino, diminuindo a sua capacidade de generalizar o conhecimento para novas instâncias (TAYLOR; NITSCHKE, 2017).

A fim de superar esse desafio, novos dados podem ser gerados a partir dos originais, mantendo os rótulos originais (tarefas de classificação), ou os adaptando (tarefa de localização). Tais transformações podem ser feitas a partir de translações, rotações, aplicação de zoom, injeção de ruído, transformações de cores, etc (SHORTEN; KHOSHGOFTAAR, 2019; TAYLOR; NITSCHKE, 2017).

Na Figura 2.10 é possível ver a imagem original em Figura 2.10a e uma nova versão que pode ser utilizada para treinamento em Figura 2.10b, onde o objeto principal pode ser identificável, mas foi aplicada rotação de 20°, zoom de 1.25x, aumento do brilho em $0,2$, contraste em $0,05$ e rotação da imagem no seu eixo horizontal.

Para vídeos, o aumento de dados pode tanto ser realizado da forma como já foi explicado, mas também pode ser feito o uso de mais quadros. Ou seja, supondo que o modelo de detecção utilize 16 quadros para realizar a classificação, então, com 32 quadros sequenciais e sem sobreposição, seria possível possuir duas entradas distintas.

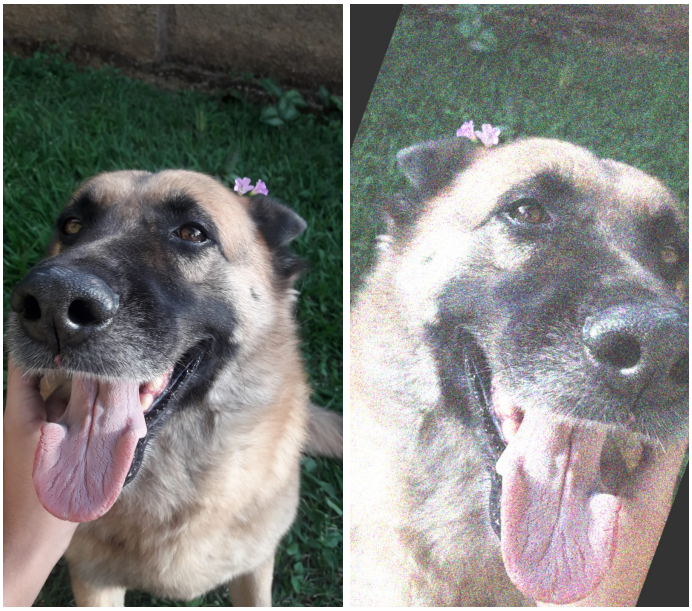
2.1.11 Generalização

Aumentar a quantidade de dados para o conjunto de treinamento, mesmo que artificialmente, também aumenta a dificuldade do modelo em encontrar padrões tão específicos para esse conjunto de forma a não ocorrer erros. Todavia é necessário garantir que uma performance similar seja alcançável para dados ainda não vistos, gerando um modelo generalizável, uma vez

Figura 2.10 – Comparação de imagem original e imagem com aumento de dados

(a) Imagem original

(b) Rotação, zoom, brilho, contraste e *flip* horizontal



MARQUES, Felipe Braz. Imagem digital. 2022. Arquivo pessoal

que os dados no mundo real dificilmente serão muito parecidos com os usados no treinamento (ZHANG et al., 2023; GOODFELLOW; BENGIO; COURVILLE, 2016).

Ao realizar o treinamento, pode ocorrer que o modelo possua uma baixa complexidade, impedindo a identificação das características dos dados fornecidos. Consequentemente, ele não irá realizar boas predições. Diz-se, então, que o modelo apresenta um subajuste, ou em inglês, *underfitting* (JABBAR; KHAN, 2015). Como exemplo, supondo um modelo complexo o suficiente para gerar apenas uma função de primeiro grau, o mesmo não conseguirá extrair padrões o suficiente para predizer corretamente os dados que foram informados e que pertençam a uma função logarítmica.

Por outro lado, caso a quantidade de dados de treinamento seja baixa, o modelo pode aprender muito bem as suas características e especificidades, se moldando extremamente bem para esse conjunto de dados. O mesmo também pode ocorrer quando o modelo é extremamente complexo e livre para aprender todas as informações e ruídos dos dados. Quando isso ocorre, os resultados para os dados de treinamento tendem a ser excelentes, mas a qualidade para dados novos pode ser prejudicada, dificultando a generalização do modelo e gerando um processo chamado de sobreajuste, ou, em inglês, *overfitting* (YING, 2019).

2.1.12 Regularização

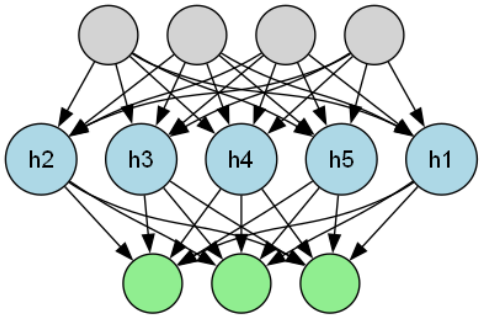
A fim de se evitar a grande variação entre os resultados obtidos no treinamento e no teste, ou seja, o *overfitting*, é comum a aplicação de técnicas de regularização a fim de diminuir a complexidade do modelo ou gerar mais desafios para o aprendizado. Assim, mesmo possuindo uma pequena perda nos resultados dos dados de treinamento, a tendência é que o modelo consiga possuir uma melhor generalização para dados ainda não vistos (GOODFELLOW; BENGIO; COURVILLE, 2016).

2.1.12.1 *Dropout*

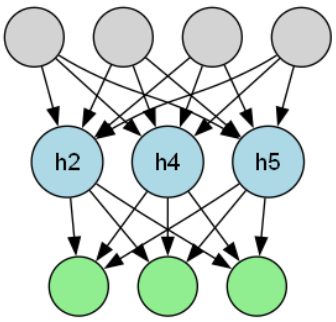
Como já mencionado anteriormente, uma das formas para se evitar o *overfitting* é a partir da construção de modelos menores. Por outro lado, outra forma seria de utilizar dados com ruídos, o que dificultaria o processo do modelo se ajustar muito para uma sequência de dados (ZHANG et al., 2023).

Para atingir esses objetivos, é possível realizar o desligamento de alguns neurônios aleatórios durante o período de treinamento. Assim, o modelo ficará ligeiramente menor e os neurônios que sobraram deverão desenvolver uma redundância, uma vez que não poderão mais "confiar"nos dados daquele que foi desligado (ZHANG et al., 2023).

Figura 2.11 – Exemplo de aplicação do *dropout*.



(a) Rede original



(b) Rede após o *dropout*

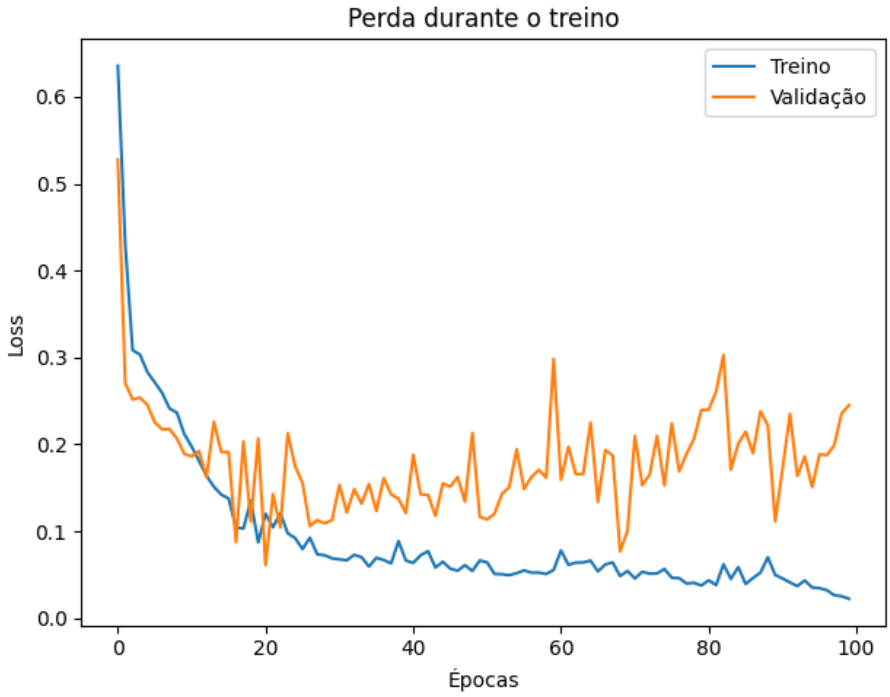
Fonte: *Imagem do autor*.

A Figura 2.11 mostra o exemplo de antes e depois de uma rede arbitrária passar pelo *dropout*. A Figura 2.11a mostra como é a arquitetura original da rede, enquanto a Figura 2.11b mostra o resultado da rede após aplicar um *dropout* aleatório para desativar 40% dos neurônios.

2.1.12.2 Paragem antecipada

Durante o treinamento, é comum que o erro de validação decaia junto com o de treinamento. Todavia, caso ocorra um *overfitting*, o erro da validação começará a crescer, como ocorre na Figura 2.12, onde o erro para de decair após a 20ª época e começa a crescer.

Figura 2.12 – *Overfitting* do modelo.



Fonte: *Imagem do autor*.

Após ocorrer o sobreajuste, a generalização do modelo fica comprometida e, dificilmente ocorrerão melhoras que permitirão que o modelo aprenda. Assim, tempo e poder computacional podem economizados ao identificar qual é esse momento. Tal técnica é conhecida como paragem antecipada, e pode ser implementada de diversas formas, entre elas, um monitor para a função de perda, onde, após algumas iterações do treinamento, caso não ocorra melhoras, encerra o processo, pois assume-se que não haverá mais melhorias (GOODFELLOW; BENGIO; COURVILLE, 2016; ZHANG et al., 2023).

2.1.13 Otimização de hiper-parâmetros

Ao realizar o treinamento de um modelo, é necessário definir a quantidade de neurônios em cada camada, a porcentagem de neurônios afetados no *dropout*, a taxa de aprendizado e outros parâmetros que não podem ser aprendidos durante o treinamento, os chamados hiper-parâmetros.

A escolha desses valores pode ser o responsável pela convergência, ou não, do modelo. Porém uma boa definição dos mesmos pode ser uma tarefa demorada e difícil, uma vez que não é possível saber de antemão se as escolhas estão no caminho certo ou não (ZHANG et al., 2023; ASZEMI; DOMINIC, 2019).

A fim de automatizar esse processo, diferentes métodos são propostos, sendo um dos mais básicos, a pesquisa aleatória. Nela, dado um espaço de possibilidades, é escolhido de forma aleatória um valor para ser utilizado como o hiper-parâmetro da iteração atual da busca (ZHANG et al., 2023).

Outra forma de otimização é a busca em grade, um algoritmo exaustivo que cobrirá todas as possibilidades de hiper-parâmetros definidos. Sendo uma de suas principais características a capacidade de paralelização (ASZEMI; DOMINIC, 2019).

Já a Otimização Bayesiana consiste em dois componentes que trabalham em conjunto: (i) um modelo de Processo Gaussiano, que aproxima a função objetivo e fornece tanto uma média prevista quanto uma estimativa de incerteza para cada ponto no domínio; e (ii) uma função de aquisição, que decide onde avaliar a seguir, equilibrando exploração e exploração. O algoritmo começa avaliando a função objetivo em um conjunto inicial de pontos e ajusta o modelo de Processo Gaussiano a esses dados. Em seguida, utiliza a função de aquisição para selecionar o próximo ponto a ser avaliado. Esse processo de avaliação, atualização do modelo e seleção de pontos é repetido iterativamente até que um critério de parada seja atendido (FRAZIER, 2018). Esse método evita efetivamente regiões não promissoras do espaço de busca, ao contrário da busca aleatória, e é mais eficiente computacionalmente do que a busca exaustiva em grade (YANG; LIU; WEN, 2024).

2.1.14 Métricas de avaliação

Ao criar um modelo de aprendizado de máquina, é importante realizar uma avaliação do mesmo para identificar se ele é promissor, e tentar otimizá-lo, ou para mudar de abordagem (RAINIO; TEUHO; KLÉN, 2024).

Durante o treinamento, é utilizado um conjunto de dados sobre o qual o modelo aprenderá as informações necessárias e se ajustará. Por outro lado, para verificar a eficácia do mesmo, é necessário utilizar dados que não foram utilizados para o ajuste dos parâmetros, sendo este conhecido como o conjunto de validação ou teste (RAINIO; TEUHO; KLÉN, 2024).

Em problemas de classificação binária, um limiar é definido para separar as duas classes, em geral, caso o valor predito seja maior que 0,5, será da classe a e será da classe b caso contrário (RAINIO; TEUHO; KLÉN, 2024). Dessa forma, há somente quatro tipos de resultados esperados, sendo eles:

- Verdadeiro-positivo (*True positive* - TP): O dado rotulado como verdadeiro foi classificado

corretamente;

- Verdadeiro-negativo (*True negative* - TN): O dado rotulado como falso foi classificado corretamente;
- Falso-positivo (*False positive* - FP): O dado rotulado como falso foi classificado como verdadeiro;
- Falso-negativo (*False negative* - FN): O dado rotulado como verdadeiro foi classificado como falso.

Dessa forma, tais informações podem ser sumarizadas em uma tabela conhecida como matriz de confusão (BRUCE; BRUCE; GEDECK, 2020), como pode ser vista na Figura 2.13.

Figura 2.13 – Matriz de confusão

		Valor esperado	
		Verdadeiro	Falso
Valor predito	Verdadeiro	TP	FP
	Falso	FN	TN

Fonte: *Imagem do autor*.

A partir dela, é possível calcular quantas vezes o modelo fez uma predição correta, ou seja, de todos os dados, quantos deles eram TP e TN, determinando a acurácia (RAINIO; TEUHO; KLÉN, 2024), que pode ser matematicamente escrita por:

$$Acc = \frac{TP + TN}{TP + TN + FP + FN}. \tag{2.24}$$

Entretanto, caso o conjunto de dados seja muito desbalanceado, o modelo pode atribuir um único valor para qualquer tipo de informação e conseguir uma alta acurácia. Dessa forma, visando evitar esse problema, métricas como precisão e revocação são utilizadas (GOODFELLOW; BENGIO; COURVILLE, 2016).

A precisão é utilizada para saber quantas vezes os dados classificados como reais de fato eram verdadeiros (RAINIO; TEUHO; KLÉN, 2024), sendo definida pela equação:

$$P = \frac{TP}{TP + TN}. \tag{2.25}$$

A revocação (também chamada de *recall* ou sensitividade), indica a porcentagem dos dados verdadeiros que foram classificados corretamente (RAINIO; TEUHO; KLÉN, 2024), sendo definida pela equação:

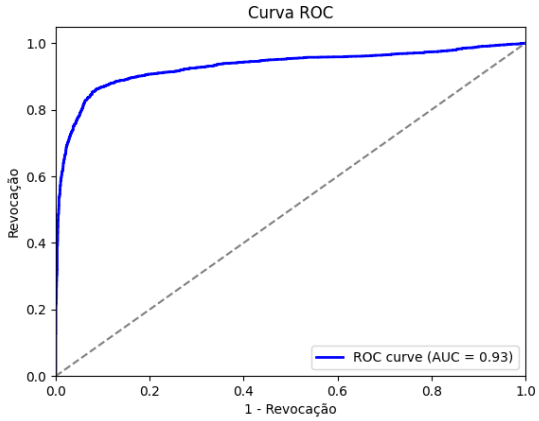
$$R = \frac{TP}{TP + FN} \tag{2.26}$$

Por causa da natureza da precisão e do *recall*, o aumento de um implica na diminuição do outro, ocorrendo um *tradeoff*, que pode ser interpretado pela curva de Característica de

Operação do Receptor, ou em inglês *Receiver Operating Characteristics* (ROC), onde um gráfico é plotado com a taxa de falso positivo (1-*recall*) no eixo das abscissas e o *recall* no eixo das ordenadas. Assim, calcula-se diferentes *recall* e taxas de falso positivo para diferentes limiares de classificação, plotando-os no gráfico (RAINIO; TEUHO; KLÉN, 2024).

A Figura 2.14 exibe uma curva ROC para dados aleatórios em diferentes limiares de classificação.

Figura 2.14 – Curva ROC



Fonte: *Imagem do autor*.

Calculando a Área Sob a Curva, ou em inglês AUC, para a curva ROC é possível encontrar um valor numérico que indica a capacidade do modelo de realizar uma boa separação dos valores entre as diferentes classes. Dessa forma, quanto maior o AUC, melhor o modelo, pois possui uma baixa taxa de classificação errônea.

2.1.15 *Deepfake*

Deepfake é uma combinação dos termos '*deep learning*' e '*fake*', e refere-se a uma tecnologia que utiliza redes neurais profundas para gerar arquivos de mídia altamente convincentes e realistas. Ele é utilizado, principalmente, para a manipulação de rostos e vozes, gerando fotos, vídeos e arquivos de áudio (MIRSKY; LEE, 2021).

A Figura 2.15 refere-se a um quadro de um dos vídeos da base de dados para detecção de *deepfakes*, a Celeb-DF (LI et al., 2020). Onde na imagem à esquerda há o rosto do ator Brad Pitt enquanto à direita há uma imagem em que o seu rosto está diferente, pois ele foi modificado por um método de *deepfake*.

Segundo Heidari et al. (2023), existem quatro grandes categorias de modificações faciais para *deepfakes*, sendo elas:

- **Troca de expressões**: Modifica as expressões faciais de uma pessoa, transformando uma expressão de tristeza em felicidade, por exemplo.

Figura 2.15 – Imagem real à esquerda e *deepfake* à direita



Fonte: (LI et al., 2020).

- **Geração completa da face**: Todo o rosto do indivíduo é gerado artificialmente.
- **Manipulação de atributos**: Mudanças no formato e cor do cabelo, olhos, pele, etc.
- **Troca de identidade**: Troca da face de uma pessoa origem por a de uma pessoa alvo.

Mirsky e Lee (2021) explicam a geração de *deepfakes* por um *pipeline* com três grandes etapas: pré-processamento, geração de dados e pós-processamento.

No pré-processamento, ocorre uma detecção e localização do rosto, que é extraído. Em seguida, são extraídas informações intermediárias, como a localização de pontos de interesse (olhos, boca, nariz, etc.), criação de um modelo 3D do rosto a partir da imagem 2D fornecida, utilização de mapas de UV, mapas de profundidade, etc.

Na parte de geração, uma rede neural recebe as informações extraídas do processo anterior, utilizando-as para realizar as modificações desejadas. Tal objetivo pode ser alcançado a partir do uso de arquiteturas *encoder-decoders*, modificando as informações de expressão, rosto ou outras características presentes no espaço latente, com o uso de GAN, a fim de gerar um rosto muito realista, ou com redes neurais recorrentes para gerar vídeos.

Por fim, a imagem do novo rosto é mesclada com a do vídeo original, corrigindo texturas, iluminações, ruídos e pose, a fim de que a imagem se pareça o mais real possível.

2.2 Trabalhos Relacionados

O avanço da inteligência artificial na geração e modificação de vídeos trouxe riscos significativos, tornando essencial o desenvolvimento de métodos eficazes para a detecção desse tipo de conteúdo. Nesse contexto, ao analisar diferentes abordagens de pesquisa sobre o tema, Heidari et al. (2023) observaram que a maior parte dos esforços concentram-se na detecção de

deepfakes em vídeos, seguida pela análise de conteúdos multimídia e, por último, de arquivos exclusivamente em áudio. Consequentemente, as CNNs se tornam uma das estratégias mais exploradas, uma vez que a maioria das pesquisas baseiam-se no uso de imagens.

O trabalho de Khan e Alam (2022) utiliza um conjunto de dados de imagens estáticas reais e falsas ao passo em que explora diferentes modelos de CNN para as predições, como a InceptionResNetV2, DenseNet201, ResNet152V2 e InceptionV3. Todos os modelos foram treinados sobre as mesmas condições de divisão de dados, taxa de aprendizagem e número de épocas, permitindo uma comparação entre eles e evidenciando a qualidade da InceptionResNetV2 em relação às demais ao obter uma acurácia de 99,86%. Por fim, para transparecer o modelo, foi utilizada uma técnica de *Explainable AI* com o *framework* LIME, permitindo interpretar visualmente quais regiões da imagem influenciaram a decisão do modelo. Embora não contribua para o aumento da acurácia, essa explicabilidade aumenta a confiança no sistema por parte dos usuários.

Em relação à detecção de *deepfakes* somente com o uso de vídeos, em (HERNANDEZ-ORTEGA et al., 2022) foi utilizado um modelo de redes neurais capaz de predizer os batimentos cardíacos de um indivíduo em vídeo para detectar a presença ou ausência de *deepfake*. Tal modelo foi proposto seguindo a hipótese de que, como objetos 3D não possuem características fisiológicas, há uma grande chance de que os modelos geradores de *deepfake* também não deem relevância para tais especificidades. Avaliando o modelo no nível de quadros foi possível obter uma acurácia de 98,7% na base de dados Celeb-DF (LI et al., 2020). Todavia, combinando os resultados em nível de quadros, porém com uma janela temporal, foi possível obter 99,47% de acurácia tanto para a média quanto para a mediana das predições realizadas para um período de sete segundos. Porém, utilizando um método onde sabe-se previamente a distribuição dos vídeos reais e falsos, obteve-se 100% de acurácia com seis segundos.

O estudo de Masud et al. (2023) enfrenta o problema de desbalanceamento da base Celeb-DF, que possui cerca de $10\times$ mais vídeos falsos que reais. Para mitigar esse desequilíbrio, os autores geraram vídeos artificiais de 15 segundos a partir de segmentos aleatórios de 3 segundos dos vídeos originais. Esse novo conjunto foi dividido em 80% para treino e 20% para teste, e 20 quadros uniformemente amostrados de cada vídeo foram utilizados como entrada do modelo. A arquitetura proposta é composta por uma VGG16 pré-treinada, envolta por uma camada *TimeDistributed* (que compartilha pesos entre os quadros), seguida por uma camada de global average pooling, uma *Long Short-term Memory* (LSTM) com 128 unidades, uma camada densa com 256 neurônios e ativação ReLU, e uma camada de saída sigmoide para classificação binária. O modelo alcançou acurácia de 99,24% e AUC de 99,52%, destacando-se pela alta performance e baixo custo computacional — sendo até 152 vezes mais leve que modelos concorrentes. No entanto, o trabalho apresenta uma limitação metodológica importante: não está claro se houve separação completa entre os trechos usados no treino e no teste. Caso contrário, pode haver vazamento de dados, o que comprometeria a validade dos resultados ao privilegiar a memorização

em vez da generalização.

Com o objetivo de empregar técnicas mais recentes de extração de características em imagens, Ganguly et al. (2022) propuseram uma abordagem híbrida combinando dois modelos distintos: a Xception, uma CNN utilizada como *backbone* para capturar características globais, e um *Vision Transformer* (ViT), responsável por identificar padrões locais e detectar incongruências entre regiões distantes da imagem. Ambas as arquiteturas são executadas em paralelo, e suas representações são posteriormente concatenadas para gerar uma predição unificada a partir de uma única imagem de entrada. Essa estratégia demonstrou alta eficácia, alcançando 97% de acurácia na base FaceForensics++, que foi treinada com imagens sem considerar o contexto temporal dos vídeos. Entretanto, na base *Deepfake Image Dataset*, embora a abordagem híbrida tenha obtido 95,42% de acurácia, o modelo treinado exclusivamente com a Xception apresentou desempenho semelhante, com 95,20% de acurácia. Esse resultado levanta questionamentos sobre a real necessidade de uma arquitetura tão robusta, considerando que um modelo mais simples é capaz de atingir resultados próximos.

Reis e Ribeiro (2024) propõem uma abordagem onde deseja-se verificar a veracidade de um vídeo onde já se conhece a identidade da pessoa retratada e possui arquivos de mídia reais da mesma. Dessa forma, 50 quadros são extraídos do vídeo questionado e mais 50 quadros são extraídos de 9 a 11 vídeos autênticos. Em seguida, essas informações são passadas para uma rede ResNet100 treinada sobre o *dataset* MS1Mv2. Dentre todos as imagens apresentadas no modelo, são feitas diferentes combinações entre elas a fim de se calcular a similaridade entre os rostos questionados e os reais. Com isso, 25 mil pontuações são geradas, e a maior delas é utilizado em um classificador binário que decidirá a veracidade do conteúdo. A metodologia apresentada é simples de ser entendida e gera bons resultados, como pode ser observado na base de dados Celeb-DF v2, onde foi obtido um AUC de 0,994 e uma acurácia de 97,5%.

Utilizando sistemas de atenção, Khormali e Yuan (2021) propõem um modelo dividido em três componentes principais, a localização da face e o pré-processamento, onde as faces de 20% dos quadros são extraídas utilizando o *RetinaFace*; aplicação de uma rede convolucional para a extração de *feature maps*, combinando-as com mapas de atenção por multiplicação ponto a ponto & *data augmentation* baseado em atenção, aplicando duas técnicas, a primeira, chamada de *Face Close-UP*, um mapa de atenção escolhe uma região do rosto que parece ser mais importante e a extrai, gerando uma imagem somente com essa área, já a outra, chamada de *Face Shut-Off*, oculta essas regiões consideradas mais importantes. Assim, uma técnica aprofunda a análise local, enquanto a outra incentiva a análise de outras partes. Dessa forma, para a base de dados Celeb-DF v2, o modelo atingiu AUC de 98,65% com a ResNet, 97,84% com a Xception, 98,01% com a VGG e 94,76% com a MobileNet.

O CLIP é um modelo que recebe uma imagem como entrada e produz um texto descrevendo o que está sendo exibido. Ele utiliza em seu backbone o encoder CLIP ViT-L/14, que consegue generalizar bem para diferentes *datasets*. Utilizando-o, Yermakov, Cech e Matas (2025)

congela todos os seus pesos durante o treinamento e utiliza seus tokens de classificação para um classificador binário decidir se o vídeo é real ou apresenta *deepfake* a partir de 32 quadros do vídeo. O modelo foi treinado na base de dados FaceForensics++ e foi avaliado em diferentes outros *datasets*, como a Celeb-DF v2, onde fora utilizada a separação oficial dos dados de avaliação, onde atingiu AUC de 96,62, a DFDC que teve AUC de 87,15 e a DFD que teve AUC de 98,0.

Utilizando tanto redes convolucionais quanto Vision Transformers, Heo, Yeo e Kim (2023) propõem um modelo que, a partir de rostos extraídos com MTCNN, utiliza uma EfficientNet-B7 para extrair características locais das imagens e um ViT para capturar informações globais, concatenando ambas as saídas e processando-as por camadas de atenção. Durante o treinamento, o modelo é otimizado por destilação a partir de um professor, utilizando um *token* de classe com os rótulos verdadeiros e um *token* de destilação com as predições do professor. Dessa forma, foi possível obter um AUC de 0,978 no *dataset* DFDC e de 0,993 no Celeb-DF v2.

Explorando técnicas que não envolvem redes neurais, Chen et al. (2021) propõem um modelo baseado em *Successive Subspace Learning*. Inicialmente, são extraídas as regiões do olho esquerdo, olho direito e boca de sete quadros por vídeo. As imagens são processadas pelo módulo PixelHop++, que extrai características espectrais e espaciais locais. Em seguida, essas características são reduzidas por PCA e passadas para classificadores XGBoost individuais por canal. A decisão final é obtida por meio de um ensemble que combina as saídas das diferentes regiões e quadros. Como resultado, foi possível obter um AUC de 1.00 na base de dados UADFV, 0,9495 na Celeb-DF v1 e 0,9056 na Celeb-DF v2.

Já no espectro do áudio, Hamza et al. (2022) utilizaram o conjunto de dados Fake-or-Real (REIMAO; TZERPOS, 2019), que é dividido em quatro partes: *for-original* (arquivos originais), *for-norm* (dados normalizados e balanceados em termos de gênero), *for-2-sec* (versão truncada em dois segundos do *for-norm*) e *for-rerec* (versão regravada do *for-2-sec*, simulando chamadas telefônicas). A partir desses dados, foram extraídas 270 características dos áudios. Diferentes classificadores foram então treinados para distinguir entre arquivos autênticos e falsificados, utilizando apenas 65 características extraídas pelo método de Análise de Componentes Principais, ou em inglês *Principal Component Analysis* (PCA). Os modelos de classificação *Support Vector Machine* apresentaram os melhores desempenhos nas versões *for-2-sec* (97,59%) e *for-rerec* (98,83%). Por outro lado, para o *for-norm*, o melhor resultado foi alcançado com o *Gradient Boosting*, com acurácia de 92,63%. Em relação aos dados do for-original, nenhum classificador tradicional obteve resultados expressivos. Diante disso, foi implementada uma rede neural baseada na arquitetura VGG-16, que alcançou uma acurácia de 93%, superando os valores de estado da arte.

Outras bases de dados já apresentam informações multimodais, ou seja, possuem uma combinação de áudio e vídeo modificados, como é o caso da FakeAVCeleb (KHALID et al., 2022). Dessa forma, tentando realizar a classificação da base de dados, Shahzad et al. (2023) propõem uma análise de inconsistências entre a boca e o áudio, extraindo características de áudio

e vídeo a partir de um modelo conhecido como AV-HuBERT, que são conectados a informações extraídas apenas da região dos lábios, verificando a sincronização das informações. Por outro lado, para evitar casos em que a boca não foi modificada explicitamente, é utilizado um modelo com *Video Vision Transformer* (ViViT) para extrair características de toda a face. Assim, tal abordagem lhes rendeu o estado da arte para a base de dados ao atingir uma acurácia de 0,99 e um AUC de 0,96 para todos os dados da base de teste.

Os resultados dos trabalhos supracitados podem ser vistos na Tabela 2.1, que mostra as diferentes modalidades exploradas e os melhores resultados obtidos pelos autores em seus experimentos. Assim, o presente trabalho busca um modelo para detecção de *deepfakes* em um contexto multimodal que se inspira nas pesquisas já realizadas a fim de obter resultados competitivos ao mesmo passo em que busca alternativas computacionalmente eficientes.

Tabela 2.1 – Resultado dos trabalhos relacionados.

Trabalho	Modalidade	Tempo	Dataset	Acurácia	AUC
Khan e Alam (2022)	Imagens	Nível de imagem	140k Real and Fake Faces	99,68%	
Hernandez-Ortega et al. (2022)	Vídeo	7 segundos	Celeb-DF v2	99,47%	0,9999
Masud et al. (2023)	Vídeo	20 quadros	Celeb-DF	99,24%	
Ganguly et al. (2022)	Vídeo	Nível de imagem	FaceForensics++	97,00%	
Reis e Ribeiro (2024)	Vídeo	500 quadros	Celeb-DF v2	97,50%	0,994
Khormali e Yuan (2021)	Vídeo	20%	Celeb-DF v2		0,9865
Yermakov, Cech e Matas (2025)	Vídeo	32 quadros	Celeb-DF v2 DFDC DFD		0,9662 0,8715 0,980
Heo, Yeo e Kim (2023)	Vídeo	32 quadros	Celeb-DF v2 DFDC		0,993 0,978
Chen et al. (2021)	Vídeo	7 quadros	UADFV Celeb-DF Celeb-DF v2		1,00 0,9495 0,9056
Hamza et al. (2022)	Áudio	Não informado	Fake-or-Real	93,00%	
Shahzad et al. (2023)	Áudio/Vídeo	Não informado	FakeAVCeleb	99,00%	

Fonte: *Tabela do autor*.

3 Desenvolvimento

Neste capítulo, na Seção 3.1, será definido o conjunto de dados utilizado para o desenvolvimento da pesquisa, bem como as características do mesmo e a distribuição de seus dados. Em seguida, na Seção 3.2, será apresentada a forma como os dados serão utilizados, definindo o pré-processamento deles. Por fim, a Seção 3.3 definirá o modelo proposto que será avaliado para a base de dados escolhida, bem como a forma que seus parâmetros são definidos.

3.1 Conjunto de dados

O conjunto de dados a ser utilizado nessa pesquisa é o Celeb-DF v2 (LI et al., 2020), um conjunto de vídeos em alta qualidade com gravações de diferentes celebridades, possuindo uma ampla variação em de contextos e indivíduos. A Figura 3.1 exibe uma imagem real do conjunto *Celeb-real*, a Figura 3.2 exibe uma pessoa real do conjunto *Youtube-real* e a Figura 3.3 é de um vídeo com *deepfake* do conjunto *Celeb-synthesis*.

Figura 3.1 – Imagem real do conjunto Celeb-real.



Fonte: (LI et al., 2020)

Figura 3.2 – Imagem de uma pessoa real do conjunto Youtube-real.



Fonte: (LI et al., 2020)

Figura 3.3 – Imagem de um vídeo com *deepfake* do conjunto Celeb-synthesis.



Fonte: (LI et al., 2020)

A base de dados possui ao todo 6.479 vídeos, onde 890 são vídeos reais, sendo 300 vídeos do YouTube e 590 de celebridades. Já os outros 5.639 vídeos são vídeos falsos sintetizados a partir dos vídeos de celebridades, onde foram aplicadas técnicas de *deepfakes*, como o *face swap*.

Realizando uma análise dos vídeos, percebe-se que a taxa de quadros dos mesmos não é consistente para toda a base de dados, variando de 28,75 a 30 quadros por segundo. Em relação à duração dos vídeos, as diferenças observadas são mais consideráveis, onde o vídeo mais curto possui um único quadro e o mais longo possui 740 quadros. Considerando que o presente trabalho analisará os vídeos como uma sequência de quadros, e não como imagens individuais, tal vídeo será desconsiderado.

A duração média dos vídeos é de 379 quadros, o que corresponde a aproximadamente 12,6 segundos de duração em uma taxa de 30 quadros por segundo; todavia, há um grande desvio padrão no tamanho dos vídeos, sendo de 84,7 quadros, quase três segundos de variação.

Analisando somente os vídeos falsos, percebe-se que há uma média de 375 quadros por vídeo, com um desvio padrão de 84,8 quadros, ou seja, valores similares aos da análise do conjunto de dados completo.

Agora, analisando os 889 vídeos reais, é possível perceber que a média dos vídeos é maior, sendo de 403,6 quadros, mas o desvio padrão permanece semelhante, sendo de 86,6 quadros.

Por fim, a Tabela 3.1 resume as informações apresentadas sobre os quadros dos vídeos.

Tabela 3.1 – Informações da quantidade de quadros dos vídeos.

Conjunto de dados	**Média**	**Desvio padrão**	**Mínimo**	**Mediana**	**Máximo**
Completo	379,2	84,7	105	379,2	740
Falsos	375,4	83,8	105	375,4	740
Reais	403,6	86,6	126	403,6	740

Fonte: *Tabela do autor*

Já em análise espacial, ou seja, em relação à resolução dos vídeos, também não há uma uniformidade. Eles possuem, em média, 480 pixeis de altura e 813 pixeis de largura, com um desvio padrão de 20,7 pixeis para a altura e 125,6 para a largura, evidenciando uma grande variação das suas proporções.

Para fins de avaliação, os autores originais definiram um subconjunto de teste composto por 178 vídeos reais, correspondente a 20% do conjunto de vídeos autênticos, e 340 vídeos com *deepfakes*, correspondente a 6% dos vídeos falsos. Essa divisão será utilizada para reportar a análise experimental realizada a fim de permitir a comparação direta com outros trabalhos. Os demais vídeos serão divididos em um conjunto para treinamento e outro para validação.

3.2 Pré-Processamento dos dados

Ao se trabalhar com o processamento de vídeos, algumas abordagens podem ser exploradas. Uma delas é a utilização dos quadros que compõem o vídeo de forma isolada, sem considerar o contexto temporal. A outra consiste em utilizar uma certa quantidade de quadros de forma a preservar esse contexto, sendo esta a abordagem escolhida para esta pesquisa.

Fazendo o uso de redes neurais, é necessário garantir uma coesão no tamanho das entradas dos dados. Portanto, é imprescindível a definição de uma duração fixa para o tamanho dos vídeos, assim como a sua altura e largura.

A definição da quantidade de quadros utilizados se baseará na proposta de utilizar o equivalente a um segundo de vídeo. Como discutido na Seção 3.1, para o conjunto de dados explorado, um segundo é caracterizado por 28,5 a 30 quadros, dependendo do vídeo. Contudo, visando um menor consumo de memória computacional, optou-se pelo uso da taxa de 24 quadros por segundo, um valor amplamente difundido em diferentes mídias, principalmente na produção cinematográfica (WILCOX et al., 2015).

Com a definição prévia dos vídeos que serão usados para teste, é necessário separar os que serão utilizados para treinamento e para validação. Ademais, é importante corrigir o desbalanceamento da base de dados, onde, dos vídeos que não serão utilizados no teste, sobraram 712 vídeos autênticos e 5.299 vídeos modificados. Tal correção é importante para evitar que o modelo tenha um viés muito grande para classificar todos os vídeos como *deepfakes*, pois isso resultaria em uma acurácia alta, mas não representaria os dados corretamente.

Uma solução prática para lidar com esse desbalanceamento é a extração de múltiplos sub-clipes a partir de cada vídeo original. Inicialmente, o conjunto de validação foi configurado para conter o mesmo número de vídeos falsos do conjunto de teste, ou seja, 340. No entanto, essa configuração levou a um desequilíbrio significativo nos dados de treinamento. Por meio de inspeção manual, foi possível perceber que transferir 14 vídeos falsos do conjunto de validação para o de treinamento melhoraria o balanceamento, resultando em um conjunto de validação final com 326 vídeos falsos e 163 vídeos reais.

Visando equilibrar a quantidade de vídeos reais e falsos no conjunto de validação, dois segmentos de 24 quadros foram extraídos de cada vídeo real. Para o treinamento, foram separados 4.973 clipes de vídeos falsos e, para balancear os vídeos reais, foram extraídos nove segmentos de

24 quadros de cada um dos 549 vídeos reais, totalizando 4.932 segmentos por classe, garantindo um conjunto de treinamento mais balanceado.

É importante notar que o aumento de dados é aplicado apenas durante o treinamento e validação, e não durante os testes, de modo que não afeta as métricas finais de avaliação. Além disso, todos os segmentos não se sobrepõem e são extraídos sequencialmente, ou seja, cada segmento começa exatamente onde o anterior termina.

Para ser incluído no conjunto de validação, um vídeo precisava ter pelo menos 48 quadros, suficientes para dois segmentos de 24 quadros. Para inclusão no conjunto de treinamento, era necessário um mínimo de 216 quadros, correspondentes a nove segmentos. Sete vídeos com duração entre 48 e 215 quadros foram atribuídos exclusivamente ao conjunto de validação.

Todos os vídeos restantes foram atribuídos aleatoriamente aos conjuntos de treinamento ou validação antes da extração dos segmentos. Isso garantiu que nenhum vídeo contribuísse com clipes para ambos os conjuntos, mantendo uma separação estrita entre os dados de treinamento e validação. A Tabela 3.2 apresenta um resumo do número de vídeos incluídos em cada subconjunto do conjunto de dados.

Tabela 3.2 – Divisão dos vídeos.

	Real	Falso	Total
Treinamento	$548 \times 9 = 4.932$	4.973	9.905
Validação	$163 \times 2 = 326$	326	652
Teste	178	340	518
Total	5.436	5.639	11.075

Fonte: *Tabela do autor.*

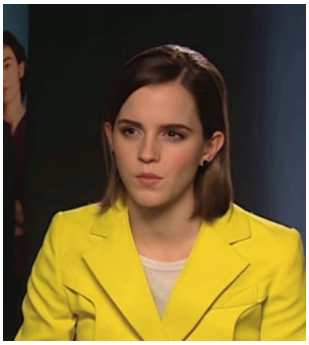
Como será utilizado o modelo Xception como extrator de características para o modelo, as dimensões das entradas deverão ser compatíveis. Assim, a resolução das imagens deverá ser de 299×299.

Porém, como analisado na Seção 3.1, os vídeos possuem uma resolução bem maior do que a esperada, uma vez que apresentam todo um contexto, e não uma região específica, como pode ser observado na Figura 3.4.

Como o foco dos *deepfakes* encontra-se na face da pessoa alvo, tal área pode ser definida como uma região de interesse. Assim, seguindo a metodologia proposta por (KHORMALI; YUAN, 2021), será utilizado o RetinFace (SERENGIL; OZPINAR, 2024) para a extração das faces dos vídeos. Como explorado por Lima et al. (2020), o RetinaFace se destaca em relação a outros modelos que se propõem a extrair faces por possuir uma alta acurácia e, consequentemente, uma baixa taxa de *False positive* (FP).

Extraindo a região de interesse, a adequação da resolução das mídias para a entrada da rede neural será facilitada. Entretanto, o modelo pode identificar o rosto como uma região muito estreita. Assim, para evitar cortes rente aos rostos, é adicionada uma margem de 10% da área

Figura 3.4 – Imagem do *dataset* com resolução 450×500.



Fonte: (LI et al., 2020).

original.

Figura 3.5 – Imagem com o rosto extraído.



Fonte: (LI et al., 2020).

Assim, aplicando o algoritmo e extraindo somente a região de interesse, obtém-se o resultado da Figura 3.5, que possui uma resolução de 152×211, que é menor do que a resolução alvo. Nesses casos, uma borda preta será adicionada ao redor da imagem, de modo que ela atinja a resolução alvo, como pode ser visto na Figura 3.6.

Figura 3.6 – Imagem com o rosto extraído e borda preta



Fonte: (LI et al., 2020).

Ao extrair o rosto da Figura 3.7a, o tamanho da imagem era de 250×364, onde a altura extrapolou os limites da resolução alvo. Dessa forma, visando evitar distorções na imagem e

manter a razão original de largura × altura, a imagem foi redimensionada para que a altura tivesse 299 pixeis. Caso a largura fosse maior, o redimensionamento seria feito com base nela. Assim, com o restante da imagem, uma borda preta é adicionada ao seu entorno para preencher o espaço faltante até os 299 pixeis da outra dimensão, gerando o resultado observado na Figura 3.7b.

Figura 3.7 – Imagem original à esquerda e rosto extraído à direita.

(a) Imagem original

(b) Extração do rosto que extrapolou os limites



Fonte: (LI et al., 2020).

3.3 Modelo proposto

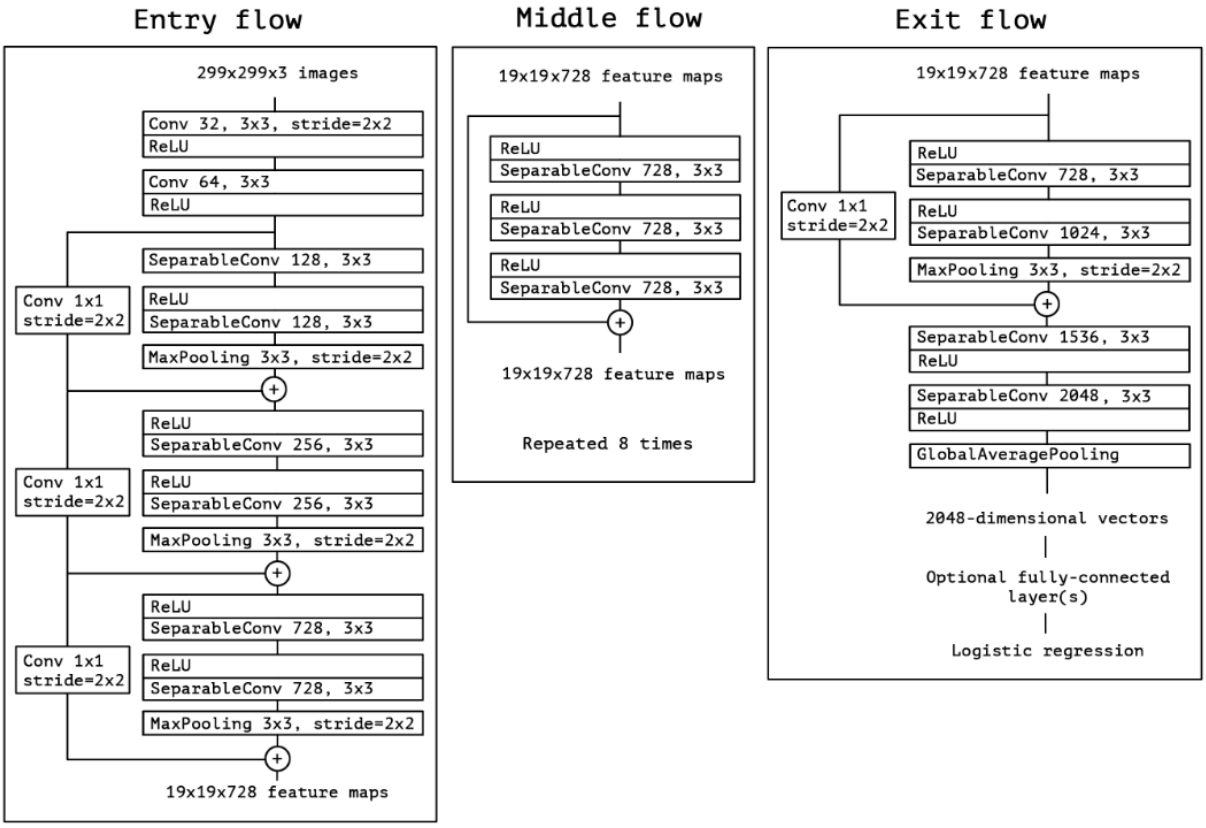
Entre os diferentes modelos bases explorados na Seção 2.2, a Xception é um dos que obtiveram o menor uso, sendo explorada em Ganguly et al. (2022) e Khormali e Yuan (2021), onde apresentaram resultados extremamente competitivos e próximos aos melhores resultados, gerando um espaço de possibilidades para melhorias.

Em sua implementação, há a presença de convolução separável em profundidade, que, diferente da convolução convencional que funciona aprendendo diferentes filtros para os diferentes canais das imagens, em tal variação, a convolução é quebrada em duas partes: 1) Convolução por profundidade, onde é aplicada uma convolução independente em cada canal de entrada; 2) Convolução ponto a ponto, onde é aplicada uma convolução de filtro 1×1 para combinar os resultados dos diferentes canais. Dessa forma, o número de parâmetros é reduzido, favorecendo o desempenho computacional (KAISER; GOMEZ; CHOLLET, 2017).

Como pode ser visto na Figura 3.8 e definida por Chollet (2017), a Xception pode ser dividida em três grandes partes que utilizam conexões residuais e convoluções separáveis em profundidade: 1) Fluxo de entrada, que recebe imagens RGB de tamanho 299×299, extraindo informações dos dados de entrada; 2) Fluxo central, que possui três blocos de ReLU seguidas por camadas de Convolução separável, possuindo, também uma conexão residual que liga a entrada ao final do terceiro bloco, repetindo oito vezes essa estrutura; 3) Fluxo de saída, que possui, em seu final, uma camada densa com regressão logística.

Como definido na Seção 3.2, os vídeos trabalhados serão compostos por 24 imagens, todavia a Xception trabalha com uma só por vez, tornando necessário uma adaptação do modelo

Figura 3.8 – Arquitetura do modelo Xception.



Fonte: (CHOLLET, 2017).

para que ele funcione com uma sequência de dados. Uma alternativa factível é a utilização de uma camada *TimeDistributed* (KERAS,), uma camada que aplica a mesma arquitetura alvo em todos os dados da sequência. Como resultado, é obtido um tensor de quatro dimensões que possui informações da relação entre espaço e tempo dos dados.

Para que esses dados sejam utilizados em um classificador, é necessário passá-los para uma camada de neurônios. Contudo, as dimensões atuais não permitem essa conexão. Assim, faz-se necessária a diminuição do tensor, podendo ser feito a partir de uma linearização dos dados, mas que geraria uma enorme quantidade de parâmetros, inviabilizando o processo. Por outro lado, é possível realizar a aplicação de um agrupamento global médio, ou seja, um *GlobalAveragePooling3D* nos dados, reduzindo as informações ao mesmo passo em que mantém informações relevantes.

Em seguida, uma camada de saída totalmente conectada é aplicada. Como destacado por Zhang et al. (2023) e explorado na Seção 2.1.13, descobrir a quantidade de neurônios adequada não é uma tarefa simples, podendo ser utilizadas técnicas de otimização de hiperparâmetros para facilitar esse processo. Dessa forma, é explorada a quantidade de neurônios para tal camada de forma que varie entre 256 a 400 com incrementos de 16, utilizando a função de ativação ReLU.

Ademais, a otimização dos hiperparâmetros de treinamento também foi considerada

essencial. Para isso, foram ajustados: (i) a taxa de aprendizado, amostrada logaritmicamente entre 1×10^{-4} e 1×10^{-2}; (ii) o parâmetro de *momentum*, amostrado logaritmicamente entre 0,87 e 0,92; e (iii) o uso da variante de *momentum* de Nesterov no otimizador *Stochastic Gradient Descent* (SGD), como uma opção booleana. Esse processo foi realizado utilizando a biblioteca KerasTuner com a técnica de otimização Bayesiana (O'MALLEY et al., 2019). O processo de otimização foi limitado a um máximo de 20 rodadas de treinamento, com parada antecipada empregada para encerrar qualquer rodada caso o erro da validação não melhorasse por sete épocas consecutivas.

A arquitetura final resultante do processo de ajuste de hiperparâmetros é apresentada na Figura 3.9. O modelo recebe tensores de entrada com dimensões $24 \times 299 \times 299 \times 3$, correspondendo a sequências de 24 quadros RGB, cada um com resolução espacial de 299×299 pixels. A dimensão temporal (24 quadros) é tratada por uma camada TimeDistributed. Essa operação gera uma saída intermediária com forma $24 \times 10 \times 10 \times 2048$.

Figura 3.9 – Modelo Final proposto.



Fonte: *Imagem do autor.*

Para agregar essas características espaço-temporais, aplica-se uma camada GlobalAveragePooling3D, reduzindo a representação para um único vetor com 2048 características. Esse vetor é então passado por uma camada densa totalmente conectada com 304 neurônios e ativação ReLU. A classificação final é realizada por uma camada de saída com dois neurônios e função de ativação softmax, permitindo ao modelo atribuir probabilidades de classe às categorias *autêntico* e *manipulado*.

Como forma de avaliar a qualidade das predições realizadas pelo modelo, propõe-se o uso das métricas de acurácia, a fim de verificar a quantidade de acertos totais que o modelo possui, e o AUC, com o objetivo de avaliar o quão bem a separação das classes está sendo feita. Ambos indicadores são utilizados nos diferentes trabalhos explorados na Seção 2.2, servindo de base para comparação com os estudos apresentados.

4 Resultados

Neste capítulo, são apresentados os resultados obtidos após a execução do treinamento do modelo proposto. Na Seção 4.1 são apresentados o ambiente onde os experimentos foram executados bem como os hiperparâmetros obtidos. Na Seção 4.2 são apresentados e analisados os resultados obtidos. Por fim, na Seção 4.3 os resultados serão explorados, analisando a forma como as classificações foram feitas e levantando hipóteses do que pode ter causado os erros obtidos.

4.1 Configuração dos experimentos

Os experimentos foram executados nas máquinas do CSI Lab, utilizando a máquina Maeve, equipada com 128 GB de RAM DDR4, GPU NVIDIA GeForce RTX 3090 com 24 GB de memória VRAM e um processador Intel(R) Core(TM) i9-10900F. A linguagem de programação escolhida foi o Python, devido a ampla quantidade de bibliotecas implementadas para a criação de redes neurais. Assim, também foram utilizadas as bibliotecas Tensorflow, na versão 2.16.1 e a biblioteca Keras-Tuner, na versão 1.4.7. Para que a compatibilidade com outros códigos ocorra, a versão utilizada do NumPy foi a 1.26.0.

Após a execução da otimização de hiperparâmetros, um novo modelo foi treinado durante 15 épocas com os valores obtidos, sendo eles:

- Número de neurônios da camada densa: 304;
- Taxa de aprendizado: $7,1030 \times 10^{-4}$;
- *Momentum*: 0,9144;
- Nesterov: `Falso`.

4.2 Resultados obtidos

Após realizar o processo de treinamento, os resultados obtidos podem ser visualizados tanto na Tabela 4.1, quanto na Figura 4.1.

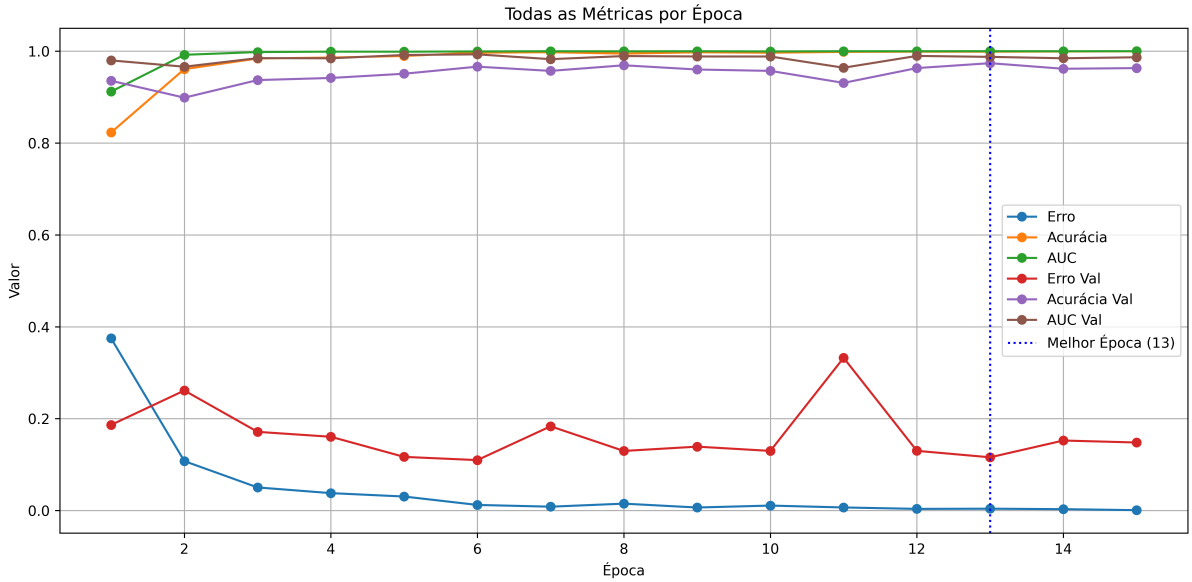
A Figura 4.2 exibe o resultado dos erros das épocas ao longo do treinamento. É possível perceber um grande decaimento dos erros de treinamento e validação nas primeiras seis épocas, sendo que, na sexta época, é onde ocorre o menor erro de validação em todo o processo de treinamento. A partir desse ponto, o erro de treinamento passa a se estabilizar e a decair mais devagar. Por outro lado, é possível perceber um alto *overfitting* do modelo nas épocas 7 e,

Tabela 4.1 – Resultados obtidos durante as 15 épocas de treinamento.

Época	Erro	Acurácia	AUC	Erro Val	Acurácia Val	AUC Val
1	0,3748	0,8230	0,9120	0,1864	0,9356	0,9799
2	0,1076	0,9610	0,9921	0,2613	0,8988	0,9662
3	0,0503	0,9841	0,9981	0,1713	0,9371	0,9849
4	0,0379	0,9861	0,9990	0,1606	0,9417	0,9845
5	0,0305	0,9896	0,9989	0,1168	0,9509	0,9917
6	0,0122	0,9967	0,9995	0,1098	0,9663	0,9931
7	0,0084	0,9978	0,9999	0,1834	0,9571	0,9826
8	0,0150	0,9947	0,9997	0,1297	0,9693	0,9894
9	0,0066	0,9976	0,9999	0,1391	0,9601	0,9885
10	0,0108	0,9967	0,9995	0,1298	0,9571	0,9884
11	0,0067	0,9985	0,9999	0,3325	0,9310	0,9640
12	0,0036	0,9989	0,9999	0,1300	0,9632	0,9897
13	0,0040	0,9989	1,0000	**0,1159**	**0,9739**	**0,9876**
14	0,0030	0,9993	0,9999	0,1524	0,9617	0,9846
15	**0,0008**	**0,9997**	**1,0000**	0,1481	0,9632	0,9867

Fonte: *tabela do autor.*

Figura 4.1 – Resultados obtidos durante 15 épocas

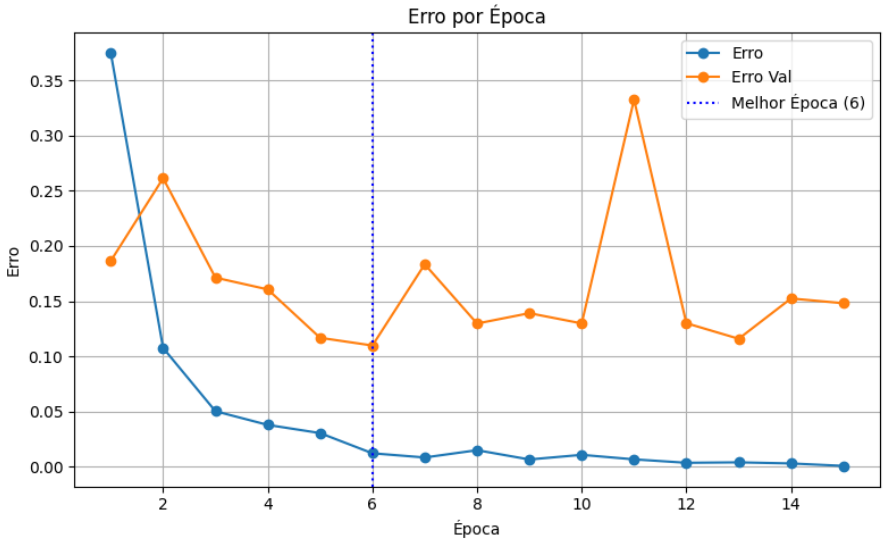


Fonte: *Imagem do autor.*

principalmente, 11, voltando a atingir valores mais baixos na época 13, que se torna a segunda época com menor erro de validação.

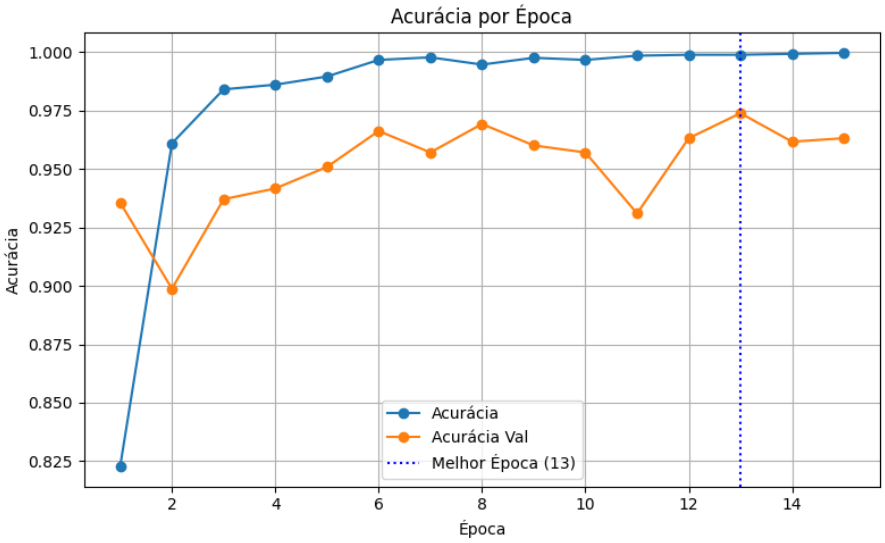
A Figura 4.3 exibe o resultado da acurácia obtida ao longo dos processos de treinamento. Analisando junto com os dados de erro, é possível perceber uma boa correlação entre eles. Ou seja, nos momentos em que o erro cai, a acurácia aumenta, e quando o erro aumenta, a acurácia cai. Tal comportamento pode ser evidenciado, principalmente, nas épocas 7 e 11, onde a acurácia cai significativamente nos momentos em que o erro aumentou. Por fim, a melhor acurácia para o

Figura 4.2 – Erros obtidos



Fonte: *Imagem do autor.*

Figura 4.3 – Acurácias obtidas



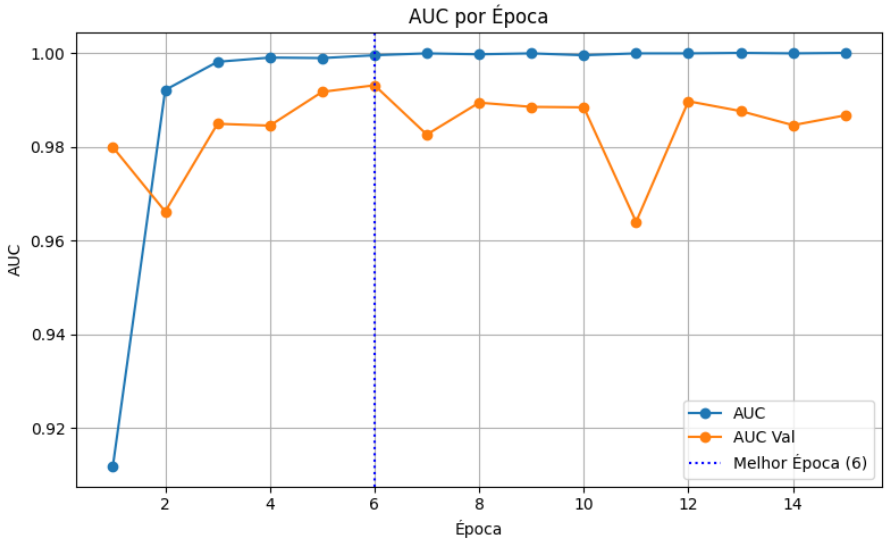
Fonte: *Imagem do autor.*

teste foi obtida na época 13. Assim, os valores obtidos nessa época foram considerados como os melhores, sendo utilizados posteriormente para a avaliação no conjunto de testes.

Diferente da acurácia, o AUC possui um comportamento um pouco diferente, como evidenciado na Figura 4.4. Para essa métrica, o melhor resultado foi atingido na época 6, justamente quando o erro começa a diminuir mais lentamente e quando ocorre o menor erro de validação. Mas seu resultado também sofre com o grande *overfitting* da época 11. E, na época 13, não consegue atingir melhores resultados assim como a acurácia.

Dessa forma, utilizando os parâmetros obtidos durante a época 13, o modelo foi avaliado no conjunto de testes onde obteve um erro de 0,0789, uma acurácia de 97,30% e um AUC de

Figura 4.4 – AUC obtidos



Fonte: *Imagem do autor.*

0,9957.

Tabela 4.2 – Comparação do modelo proposto com trabalhos que utilizaram a Celeb-DF v2, ordenados pelo AUC

Trabalho	Quadros usados	Acurácia	AUC
Chen et al. (2021)	7	Não informado	0,906
Yermakov, Cech e Matas (2025)	32	Não informado	0,966
Khormali e Yuan (2021)	Não informado	Não informado	0,987
Heo, Yeo e Kim (2023)	32	Não informado	0,9930
Reis e Ribeiro (2024)	50 + 50*9	97,50%	0,994
Modelo proposto	24	97,30%	0,9957
Hernandez-Ortega et al. (2022)	210	99,47%	0,9999

Fonte: *tabela do autor.*

Por fim, ao comparar esses resultados com aqueles extraídos dos trabalhos relacionados que também utilizam o conjunto de dados Celeb-DF v2, conforme apresentado na Tabela 4.2, é evidente que o modelo proposto alcança um desempenho competitivo em termos de acurácia e AUC.

Embora a estratégia proposta não atinja a maior acurácia entre todos os métodos relacionados no Celeb-DF v2, ela opera utilizando o segundo menor número de quadros, ou seja, com menos dados, resultando em um custo computacional potencialmente menor, ao mesmo tempo em que mantém acurácia e AUC competitivas.

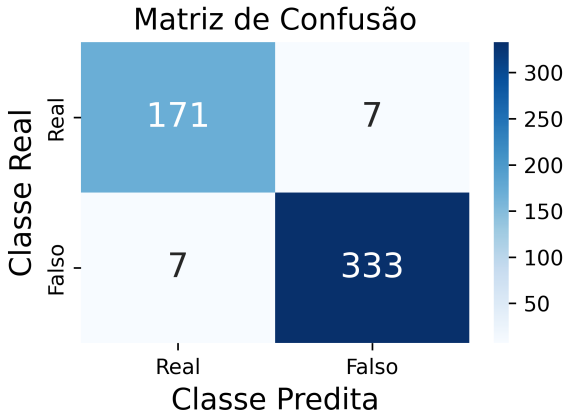
4.3 Discussão dos resultados

A Figura 4.5 apresenta a matriz de confusão resultante da avaliação no conjunto de teste. Dos 518 vídeos utilizados para avaliar o modelo, 14 foram categorizados em classes incorretas.

Dentre esses, sete vídeos reais foram classificados incorretamente como falsos, enquanto sete vídeos falsos foram classificados erroneamente como reais.

Para os vídeos reais, há uma precisão e revocação de 96,07%. Já para os vídeos falsos, há uma precisão e revocação de 97,94%. Levando em consideração o desequilíbrio significativo entre as classes no conjunto de dados, a matriz de confusão e os valores de precisão e revocação demonstram que o modelo foi capaz de aprender, de forma efetiva, padrões que diferenciam vídeos autênticos de manipulados.

Figura 4.5 – Matriz de confusão dos dados de teste



Fonte: *Imagem do autor.*

Analisando os vídeos reais que foram classificados como falsos, que possuem um quadro extraídos e apresentados na Figura 4.6, é possível notar que alguns vídeos foram redimensionados para se ajustarem ao limite de 299 pixeis da região facial ou já apresentavam resolução originalmente baixa. No primeiro caso, uma possível explicação para a classificação incorreta é que o processo de redimensionamento pode ter resultado em agregação de pixeis, obscurecendo características relevantes e dificultando a capacidade do modelo de identificar corretamente o vídeo como genuíno. No segundo caso, em que a resolução original já era baixa, o modelo pode não ter informação visual suficiente para detectar os padrões necessários à classificação correta, levando assim a previsões equivocadas.

Figura 4.6 – Reais classificados como falsos.



Fonte: *Imagem do autor.*

A Figura 4.7 apresenta um quadro de cada vídeo falso que foi incorretamente classificado como real. Nesses exemplos, observa-se que as imagens já eram menores que o limite de 299 pixeis, como evidenciado pela presença de margens pretas ao redor das regiões faciais. Essa

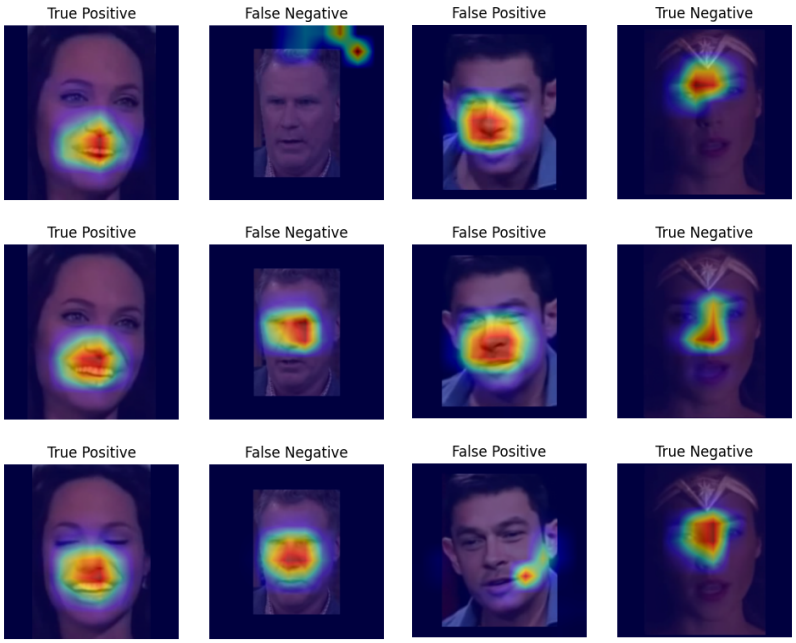
resolução reduzida provavelmente comprometeu a capacidade do modelo de detectar artefatos característicos de *deepfakes*, contribuindo para a classificação incorreta.

Figura 4.7 – Falsos classificados como reais



Fonte: *Imagem do autor.*

Figura 4.8 – Mapas de ativação do primeiro quadro (primeira linha), do quadro intermediário (segunda linha, décimo segundo quadro) e do quadro final de quatro vídeos e suas respectivas classificações. Regiões com maior relevância estão destacadas em vermelho, enquanto áreas menos significativas são representadas em azul.



Fonte: *Imagem do autor.*

A Figura 4.8 exibe os mapas de ativação Grad-CAM para o primeiro quadro, um quadro intermediário (o décimo segundo) e o quadro final de um vídeo. Esses quadros foram selecionados para ilustrar como o modelo proposto interpreta e classifica diferentes segmentos ao longo do vídeo.

Cores mais próximas do vermelho indicam uma maior relevância da região durante o processo de classificação, enquanto as áreas em azul representam regiões menos significativas. Assim, percebe-se que a análise não se concentra em uma única região, se deslocando para outras conforme o vídeo é avançado. Esse comportamento sugere uma estratégia de análise específica por quadro, em vez de uma abordagem global, o que é consistente com o fato de que as regiões manipuladas geralmente variam à medida que o vídeo avança e nem sempre permanecem as mesmas.

5 Considerações Finais

Neste capítulo, na Seção 5.1 será feita um resumo geral do trabalho evidenciando os resultados e os objetivos alcançados, bem como os resultados obtidos. Já a Seção 5.2 definirá os caminhos a serem traçados para a continuidade da pesquisa.

5.1 Conclusão

O presente trabalho teve como objetivo principal a exploração de modelos de inteligência artificial baseados em *deep learning* para classificar vídeos como autênticos ou com a presença de modificações por meio de *deepfakes*, de forma a atingir métricas competitivas em comparação com a literatura. Assim, tal desafio foi superado ao se apresentar um modelo baseado na arquitetura Xception, que obteve uma acurácia de 97,30% e um AUC de 0,9957.

Para que esse objetivo pudesse ser alcançado, foi necessária a investigação da literatura, a qual indicou que o uso de CNN é altamente eficiente na classificação de vídeos contendo *deepfakes*. Ademais, tal pesquisa também foi crucial para a escolha da base de dados utilizada, no caso, a Celeb-DF v2, bem como para a definição das estratégias de pré-processamento, nas quais as faces foram extraídas utilizando a RetinaFace.

Em seguida, a proposta de um modelo e a identificação de seus hiperparâmetros mostraram-se de extrema importância, uma vez que foi possível encontrar configurações que resultaram em excelentes métricas de avaliação. Ao serem comparados com o estado da arte, tais resultados revelam-se extremamente competitivos. Com relação ao número de quadros, o modelo proposto é aquele que requer a segunda menor quantidade, superando inclusive o modelo que utiliza a menor. Já em termos de AUC, ocupa a segunda colocação, mas com uma quantidade de dados para inferência consideravelmente inferior àquela empregada pelo modelo com os melhores resultados.

Por fim, com base nos resultados apresentados, conclui-se que foi possível atingir os objetivos propostos para esta pesquisa. Por outro lado, não se descarta a possibilidade de alcançar resultados ainda melhores na segunda etapa do trabalho, seja por meio do refinamento das técnicas de pré-processamento, seja pelo desenvolvimento de modelos mais robustos.

5.2 Trabalhos Futuros

O presente trabalho visa ser concluído durante o período vigente para a disciplina de Monografia II. Assim, a Tabela 5.1 apresenta um planejamento inicial dividido em semanas para a formulação do trabalho.

Tabela 5.1 – Separação de atividades para a Monografia II

Atividades	Semanas
Construção e avaliação de diferentes redes de classificação	01-05
Aprimoramento da rede proposta	06-10
Validação do modelo proposto	11-14
Refatoração e disponibilização do código fonte	15-16
Escrita da Monografia	05-16
Apresentação da Monografia	17

Fonte: *tabela do autor.*

Em um passo inicial, o objetivo se torna criar diferentes modelos e treiná-los, a fim de explorar diferentes arquiteturas e estratégias para a classificação de dados. Em um segundo passo, visa aprimorar a rede que obteve melhores resultados, realizando a *tunagem* de hiperparâmetros para tentar extrair resultados ainda melhores da rede considerada como mais promissora.

Com um bom modelo em mãos, o modelo será validado com o conjunto de testes, explorando as classificações erradas feitas, a fim de reconhecer padrões e propor hipóteses de futuras melhorias. O código utilizado será refatorado para facilitar a compreensão de possíveis pesquisadores futuros que possam vir a se interessar pelo processo feito, facilitando a continuação dos estudos e a comparação dos resultados obtidos.

Durante todo o processo, o presente arquivo será iterativamente modificado a fim de registrar todos os passos feitos, documentando tanto os experimentos favoráveis quanto aqueles que não são promissores.

Por fim, com todas as formalizações de códigos e documentações, será realizada, então, a apresentação da Monografia II com os resultados obtidos.

Referências

ALAGöZLü, M. Stochastic gradient descent variants and applications. 02 2022.

ASZEMI, N. M.; DOMINIC, P. D. D. Hyperparameter optimization in convolutional neural network using genetic algorithms. *International Journal of Advanced Computer Science and Applications*, Science and Information (SAI) Organization Limited, v. 10, n. 6, 2019.

BRUCE, P.; BRUCE, A.; GEDECK, P. *Practical statistics for data scientists*. 2. ed. Sebastopol, CA: O'Reilly Media, 2020.

BURKOV, A. *The hundred-page machine learning book*. [S.l.]: True Positive, 2019.

CAMPBELL, M.; HOANE, A.; HSU, F. hsiung. Deep blue. *Artificial Intelligence*, v. 134, n. 1, p. 57–83, 2002. ISSN 0004-3702. Disponível em: <https://www.sciencedirect.com/science/article/pii/S0004370201001291>.

CHEN, H.; MAGRAMO, K. *Finance worker pays out $25 million after video call with deepfake chief financial officer*. 2024. Disponível em: <https://edition.cnn.com/2024/02/04/asia/deepfake-cfo-scam-hong-kong-intl-hnk/index.html>.

CHEN, H.-S.; ROUHSEDAGHAT, M.; GHANI, H.; HU, S.; YOU, S.; KUO, C. C. J. *DefakeHop: A Light-Weight High-Performance Deepfake Detector*. 2021. Disponível em: <https://arxiv.org/abs/2103.06929>.

CHOLLET, F. *Xception: Deep Learning with Depthwise Separable Convolutions*. 2017. Disponível em: <https://arxiv.org/abs/1610.02357>.

COELHO, P. *O Alquimista*. [S.l.]: Pergaminho, 2001. 34 p. ISBN 9789727110797.

ELHARROUSS, O.; MAHMOOD, Y.; BECHQITO, Y.; SERHANI, M. A.; BADIDI, E.; RIFFI, J.; TAIRI, H. *Loss Functions in Deep Learning: A Comprehensive Review*. 2025. Disponível em: <https://arxiv.org/abs/2504.04242>.

FRAZIER, P. I. *A Tutorial on Bayesian Optimization*. 2018. Disponível em: <https://arxiv.org/abs/1807.02811>.

GANGULY, S.; GANGULY, A.; MOHIUDDIN, S.; MALAKAR, S.; SARKAR, R. Vixnet: Vision transformer with xception network for deepfakes based video and image forgery detection. *Expert Systems with Applications*, v. 210, p. 118423, 2022. ISSN 0957-4174. Disponível em: <https://www.sciencedirect.com/science/article/pii/S0957417422015251>.

GONZALEZ, R. C.; WOODS, R. E. *Digital Image Processing*. 4th. ed. [S.l.]: Pearson, 2018. ISBN 9780133356724.

GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A. *Deep Learning*. [S.l.]: MIT Press, 2016. <http://www.deeplearningbook.org>.

GOODFELLOW, I. J.; POUGET-ABADIE, J.; MIRZA, M.; XU, B.; WARDE-FARLEY, D.; OZAIR, S.; COURVILLE, A.; BENGIO, Y. *Generative Adversarial Networks*. 2014. Disponível em: <https://arxiv.org/abs/1406.2661>.

HAMZA; AMEER; JAVED; REHMAN, A. R.; IQBAL; FARKHUND; KRYVINSKA; NATALIA; ALMADHOR; S., A.; JALIL; ZUNERA; BORGHOL; ROUBA. Deepfake audio detection via mfcc features using machine learning. *IEEE Access*, v. 10, p. 134018–134028, 2022.

HEIDARI, A.; NAVIMIPOUR, N.; DAG, H.; UNAL, M. Deepfake detection using deep learning methods: A systematic and comprehensive review. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, v. 14, 11 2023.

HEO, Y.-J.; YEO, W.-H.; KIM, B.-G. Deepfake detection algorithm based on improved vision transformer. *Applied Intelligence*, v. 53, n. 7, p. 7512–7527, Apr 2023. ISSN 1573-7497. Disponível em: <https://doi.org/10.1007/s10489-022-03867-9>.

HERNANDEZ-ORTEGA, J.; TOLOSANA, R.; FIERREZ, J.; MORALES, A. Deepfakes detection based on heart rate estimation: Single- and multi-frame. In: ____. *Handbook of Digital Face Manipulation and Detection: From DeepFakes to Morphing Attacks*. Cham: Springer International Publishing, 2022. p. 255–273. Disponível em: <https://doi.org/10.1007/978-3-030-87664-7_12>.

JABBAR, H.; KHAN, R. Z. Methods to avoid over-fitting and under-fitting in supervised machine learning (comparative study). *Computer science, communication and instrumentation devices*, Res. Publ Singapore, v. 70, n. 10.3850, p. 978–981, 2015.

JIKADARA, B. *Cats and Dogs Classification Dataset*. 2023. <https://www.kaggle.com/datasets/bhavikjikadara/dog-and-cat-classification-dataset>. [Online; acesso em 05 de junho de 2025].

KAISER, L.; GOMEZ, A. N.; CHOLLET, F. *Depthwise Separable Convolutions for Neural Machine Translation*. 2017. Disponível em: <https://arxiv.org/abs/1706.03059>.

KAPLAN, A.; HAENLEIN, M. Siri, siri, in my hand: Who's the fairest in the land? on the interpretations, illustrations, and implications of artificial intelligence. *Business Horizons*, v. 62, n. 1, p. 15–25, 2019. ISSN 0007-6813. Disponível em: <https://www.sciencedirect.com/science/article/pii/S0007681318301393>.

KERAS. *Keras documentation: TimeDistributed layer — keras.io*. <https://keras.io/api/layers/recurrent_layers/time_distributed/>. [Accessed 06-08-2025].

KHALID, H.; TARIQ, S.; KIM, M.; WOO, S. S. *FakeAVCeleb: A Novel Audio-Video Multimodal Deepfake Dataset*. 2022. Disponível em: <https://arxiv.org/abs/2108.05080>.

KHAN, M.; ALAM, K. N. Detecting deepfake images using deep learning techniques and explainable ai methods. *Intelligent Automation and Soft Computing*, v. 35, 07 2022.

KHORMALI, A.; YUAN, J.-S. Add: Attention-based deepfake detection approach. *Big Data and Cognitive Computing*, v. 5, n. 4, 2021. ISSN 2504-2289. Disponível em: <https://www.mdpi.com/2504-2289/5/4/49>.

KIRBY, P. *Muharrem Ince: Turkish candidate dramatically pulls out before election*. BBC News, 2023. Disponível em: <https://www.bbc.com/news/world-europe-65560052>.

KRIZHEVSKY, A.; SUTSKEVER, I.; HINTON, G. E. Imagenet classification with deep convolutional neural networks. In: PEREIRA, F.; BURGES, C.; BOTTOU, L.; WEINBERGER, K. (Ed.). *Advances in Neural Information Processing Systems*. Curran Associates, Inc.,

2012. v. 25. Disponível em: <https://proceedings.neurips.cc/paper_files/paper/2012/file/c399862d3b9d6b76c8436e924a68c45b-Paper.pdf>.

LECUN, Y.; BENGIO, Y.; HINTON, G. Deep learning. *Nature*, v. 521, n. 7553, p. 436–444, 2015. ISSN 1476-4687. Disponível em: <https://doi.org/10.1038/nature14539>.

LI, Y.; YANG, X.; SUN, P.; QI, H.; LYU, S. Celeb-df: A large-scale challenging dataset for deepfake forensics. In: *IEEE Conference on Computer Vision and Patten Recognition (CVPR)*. [S.l.: s.n.], 2020.

LIMA, O. de; FRANKLIN, S.; BASU, S.; KARWOSKI, B.; GEORGE, A. *Deepfake Detection using Spatiotemporal Convolutional Networks*. 2020. Disponível em: <https://arxiv.org/abs/2006.14749>.

MASUD, U.; SADIQ, M.; MASOOD, S.; AHMAD, M.; EL-LATIF, A. A. A. Lw-deepfakenet: a lightweight time distributed cnn-lstm network for real-time deepfake video detection. *Signal, Image and Video Processing*, v. 17, n. 8, p. 4029–4037, Nov 2023. ISSN 1863-1711. Disponível em: <https://doi.org/10.1007/s11760-023-02633-9>.

MIRSKY, Y.; LEE, W. The creation and detection of deepfakes: A survey. *ACM Computing Surveys*, Association for Computing Machinery (ACM), v. 54, n. 1, p. 1–41, jan. 2021. ISSN 1557-7341. Disponível em: <http://dx.doi.org/10.1145/3425780>.

MITCHELL, T. M.; MUNSON, E. M. *Machine learning*. [S.l.]: WCB/McGraw-Hill, 1997.

NIELSEN, M. *Neural Networks and Deep Learning*. [S.l.]: Determination Press, 2015.

O'MALLEY, T.; BURSZTEIN, E.; LONG, J.; CHOLLET, F.; JIN, H.; INVERNIZZI, L. et al. *KerasTuner*. 2019. <https://github.com/keras-team/keras-tuner>.

RADFORD, A.; NARASIMHAN, K.; SALIMANS, T.; SUTSKEVER, I. et al. Improving language understanding by generative pre-training. San Francisco, CA, USA, 2018.

RAINIO, O.; TEUHO, J.; KLÉN, R. Evaluation metrics and statistical tests for machine learning. *Sci. Rep.*, Springer Science and Business Media LLC, v. 14, n. 1, p. 6086, mar. 2024.

REIMAO, R.; TZERPOS, V. For: A dataset for synthetic speech detection. In: *2019 International Conference on Speech Technology and Human-Computer Dialogue (SpeD)*. [S.l.: s.n.], 2019. p. 1–10.

REIS, P. M. G. I.; RIBEIRO, R. O. A forensic evaluation method for deepfake detection using dcnn-based facial similarity scores. *Forensic Science International*, v. 358, p. 111747, 2024. ISSN 0379-0738. Disponível em: <https://www.sciencedirect.com/science/article/pii/S0379073823001974>.

ROSENBLATT, F. The perceptron: a probabilistic model for information storage and organization in the brain. *Psychological review*, American Psychological Association, v. 65, n. 6, p. 386, 1958.

RUMELHART, D. E.; HINTON, G. E.; WILLIAMS, R. J. Learning representations by back-propagating errors. *Nature*, v. 323, n. 6088, p. 533–536, Oct 1986. ISSN 1476-4687. Disponível em: <https://doi.org/10.1038/323533a0>.

RUSSELL, S. J.; NORVIG, P. *Artificial Intelligence: A modern approach*. [S.l.]: Pearson, 2021.

SAMUEL, A. L. Some studies in machine learning using the game of checkers. *IBM Journal of Research and Development*, v. 3, n. 3, p. 210–229, 1959.

SERENGIL, S.; OZPINAR, A. A benchmark of facial recognition pipelines and co-usability performances of modules. *Journal of Information Technologies*, Gazi University, v. 17, n. 2, p. 95–107, 2024. Disponível em: <https://dergipark.org.tr/en/pub/gazibtd/issue/84331/1399077>.

SHAHZAD, S. A.; HASHMI, A.; PENG, Y.-T.; TSAO, Y.; WANG, H.-M. *AV-Lip-Sync+: Leveraging AV-HuBERT to Exploit Multimodal Inconsistency for Video Deepfake Detection*. 2023. Disponível em: <https://arxiv.org/abs/2311.02733>.

SHORTEN, C.; KHOSHGOFTAAR, T. M. A survey on image data augmentation for deep learning. *Journal of Big Data*, v. 6, n. 1, p. 60, Jul 2019. ISSN 2196-1115. Disponível em: <https://doi.org/10.1186/s40537-019-0197-0>.

SILVERMAN, C. *How To Spot A Deepfake Like The Barack Obama–Jordan Peele Video*. 2018. Disponível em: <https://www.buzzfeed.com/craigsilverman/obama-jordan-peele-deepfake-video-debunk-buzzfeed>.

SOMERS, J. *The Pastry A.I. That Learned to Fight Cancer*. 2021. Disponível em: <https://www.newyorker.com/tech/annals-of-technology/the-pastry-ai-that-learned-to-fight-cancer>.

STEWART, J. *Cálculo, Volume 2*. 8. ed. São Paulo: Cengage Learning, 2017. ISBN 9788522129971.

TAYLOR, L.; NITSCHKE, G. *Improving Deep Learning using Generic Data Augmentation*. 2017. Disponível em: <https://arxiv.org/abs/1708.06020>.

VRBANčIč, G.; PODGORELEC, V. Transfer learning with adaptive fine-tuning. *IEEE Access*, v. 8, p. 196197–196211, 2020.

WILCOX, L. M.; ALLISON, R. S.; HELLIKER, J.; DUNK, B.; ANTHONY, R. C. Evidence that viewers prefer higher frame-rate film. Association for Computing Machinery, New York, NY, USA, v. 12, n. 4, sep 2015. ISSN 1544-3558. Disponível em: <https://doi.org/10.1145/2810039>.

YANG, K.; LIU, L.; WEN, Y. The impact of bayesian optimization on feature selection. *Scientific Reports*, v. 14, n. 1, p. 3948, Feb 2024. ISSN 2045-2322. Disponível em: <https://doi.org/10.1038/s41598-024-54515-w>.

YERMAKOV, A.; CECH, J.; MATAS, J. *Unlocking the Hidden Potential of CLIP in Generalizable Deepfake Detection*. 2025. Disponível em: <https://arxiv.org/abs/2503.19683>.

YING, X. An overview of overfitting and its solutions. *Journal of Physics: Conference Series*, IOP Publishing, v. 1168, n. 2, p. 022022, feb 2019. Disponível em: <https://dx.doi.org/10.1088/1742-6596/1168/2/022022>.

ZHANG, A.; LIPTON, Z. C.; LI, M.; SMOLA, A. J. *Dive into Deep Learning*. [S.l.]: Cambridge University Press, 2023. <https://D2L.ai>.

ŁABUZ, M.; NEHRING, C. On the way to deep fake democracy? deep fakes in election campaigns in 2023. *European Political Science*, Apr 2024. ISSN 1682-0983. Disponível em: <https://doi.org/10.1057/s41304-024-00482-9>.